



无监督（聚类）算法

Unsupervised Learning (Clustering) Algorithms



授课对象：计算机科学与技术专业 二年级

课程名称：人工智能（专业必修）

节选内容：第六章 机器学习

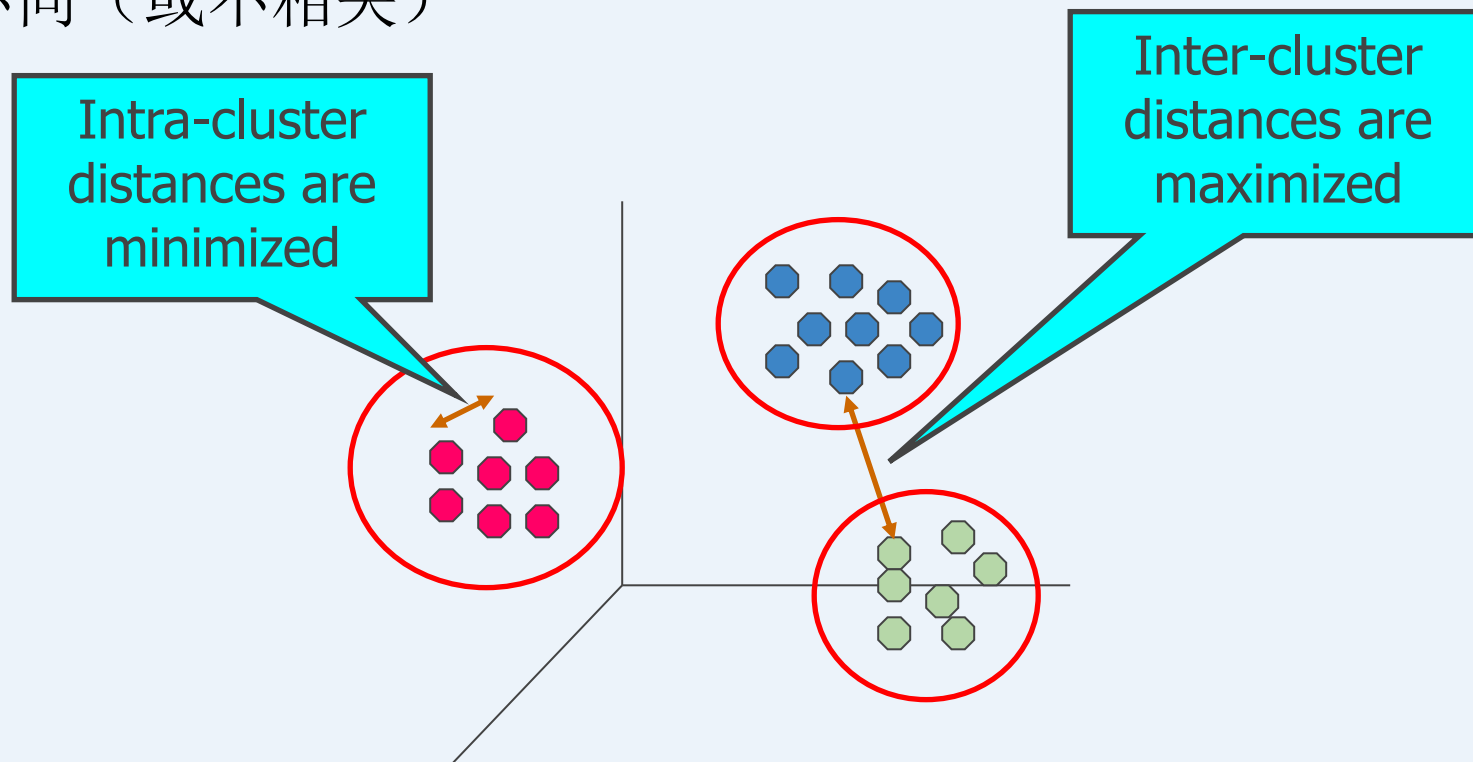
课程学分：3学分





什么是聚类分析?

Finding groups of objects such that the objects in a group will be similar (or related) to one another and different from (or unrelated to) the objects in other groups
寻找一组对象，使得同一组内的对象彼此相似（或相关），而与不同组内的对象不同（或不相关）





聚类分析的应用

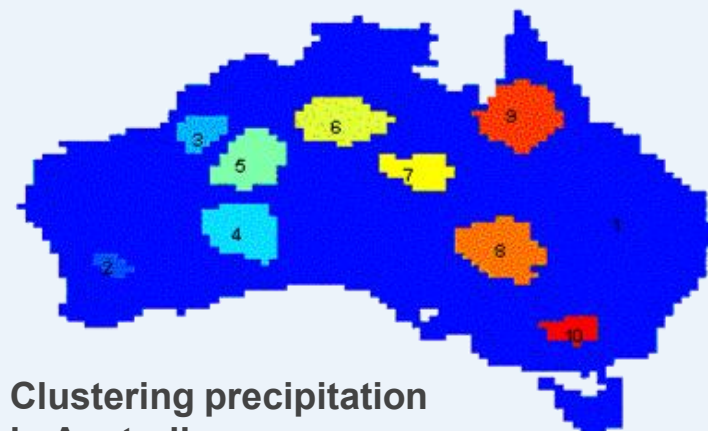
● Understanding

- 将相关文档分组以便浏览
- 将具有相似功能的基因和蛋白质分组
- 将价格波动相似的股票分组

● Summarization

- 减少大型数据集的规模

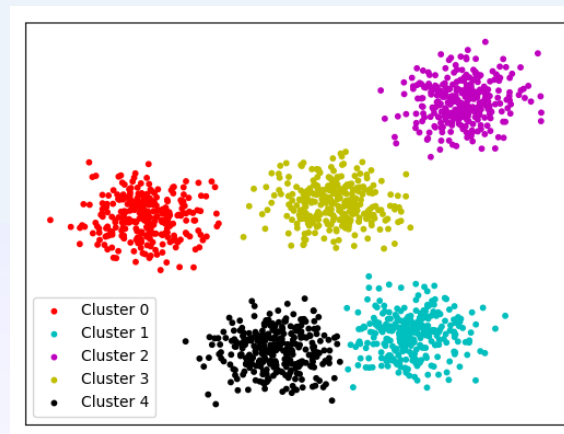
	<i>Discovered Clusters</i>	<i>Industry Group</i>
1	Applied-Matl-DOWN, Bay-Network-DOWN, 3-COM-DOWN, Cabletron-Sys-DOWN, CISCO-DOWN, HP-DOWN, DSC-Comm-DOWN, INTEL-DOWN, LSI-Logic-DOWN, Micron-Tech-DOWN, Texas-Inst-DOWN, Tellabs-Inc-DOWN, Natl-Semiconduct-DOWN, Oracl-DOWN, SGI-DOWN, Sun-DOWN	Technology1-DOWN
2	Apple-Comp-DOWN, Autodesk-DOWN, DEC-DOWN, ADV-Micro-Device-DOWN, Andrew-Corp-DOWN, Computer-Assoc-DOWN, Circuit-City-DOWN, Compaq-DOWN, EMC-Corp-DOWN, Gen-Inst-DOWN, Motorola-DOWN, Microsoft-DOWN, Scientific-Atl-DOWN	Technology2-DOWN
3	Fannie-Mae-DOWN, Fed-Home-Loan-DOWN, MBNA-Corp-DOWN, Morgan-Stanley-DOWN	Financial-DOWN
4	Baker-Hughes-UP, Dresser-Inds-UP, Halliburton-HLD-UP, Louisiana-Land-UP, Phillips-Petro-UP, Unocal-UP, Schlumberger-UP	Oil-UP



Clustering precipitation
in Australia

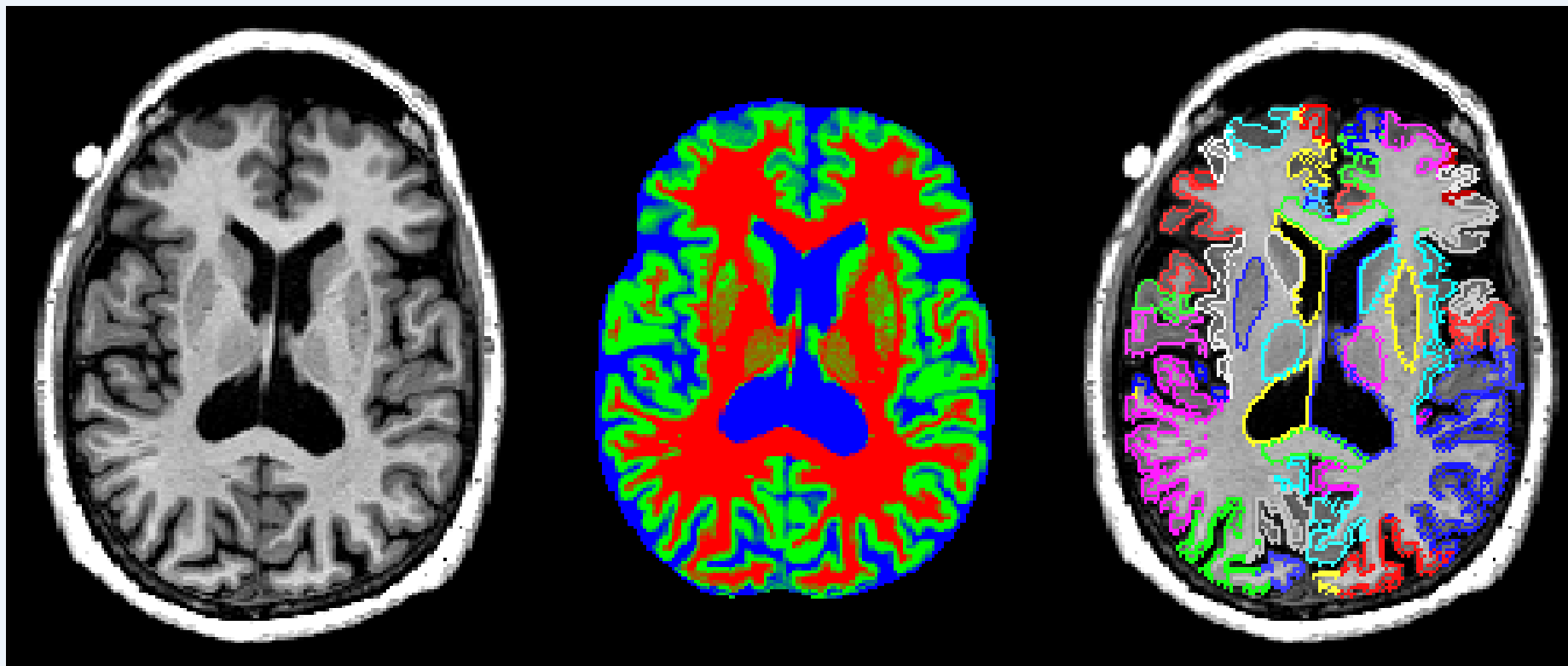


聚类：社交网络





聚类：图像分割





聚类：视频检测

TVCP



Frame #50
F-m.=0.9630

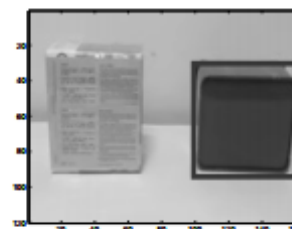


Frame #70
F-m.=0.9752

Nonnegative PARAFAC



Frame #50
F-m.=0.9209



Frame #70
F-m.=0.9512

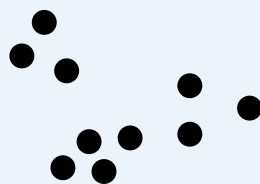
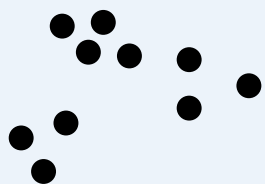


什么不是聚类分析?

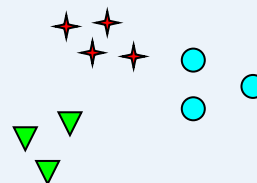
- Supervised classification
 - Have class label information
- Simple segmentation
 - Dividing students into different registration groups alphabetically, by last name



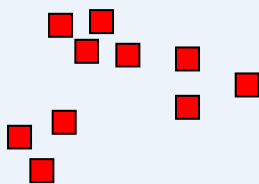
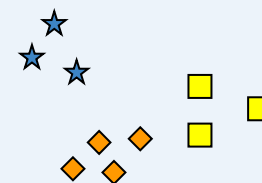
簇的概念：二义性



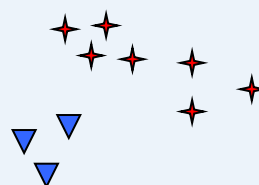
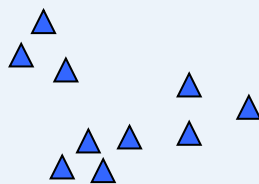
How many clusters?



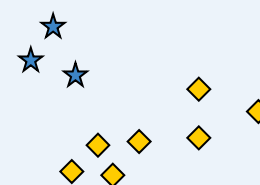
Six Clusters



Two Clusters



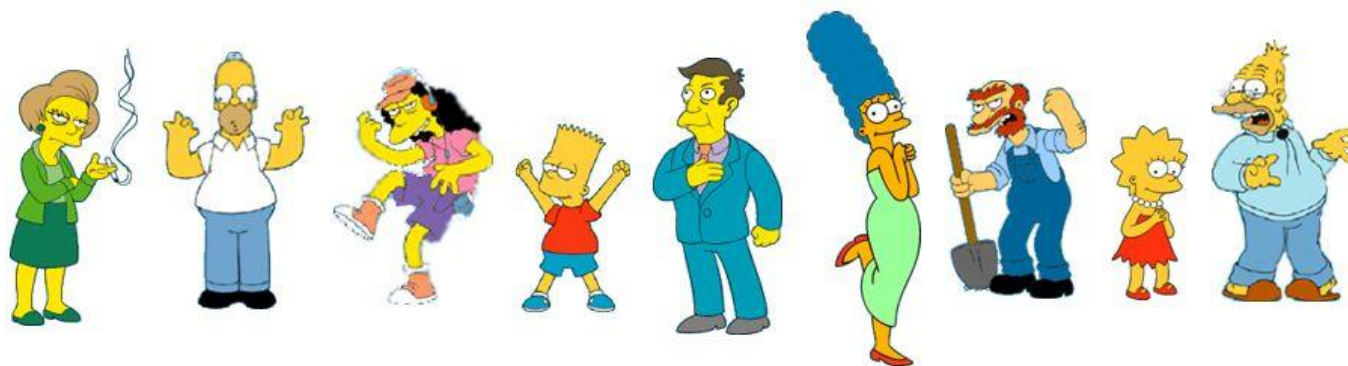
Four Clusters





簇的概念：二义性

What is a natural grouping among these objects?



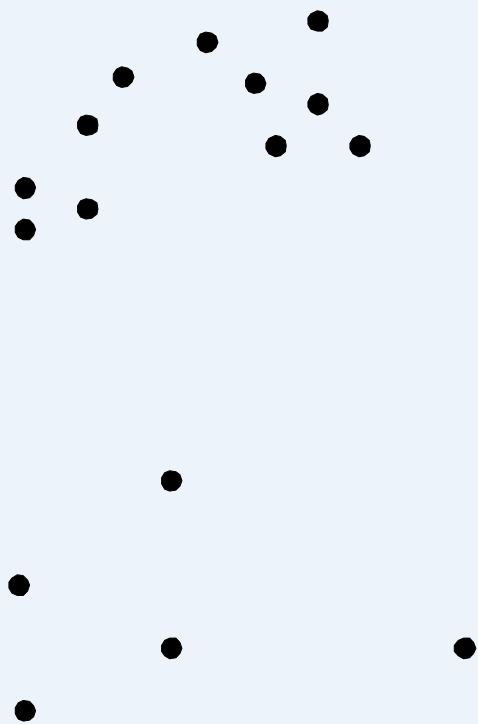


聚类(Clustering)类型

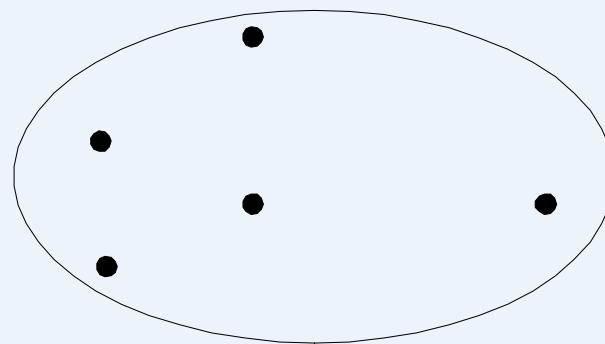
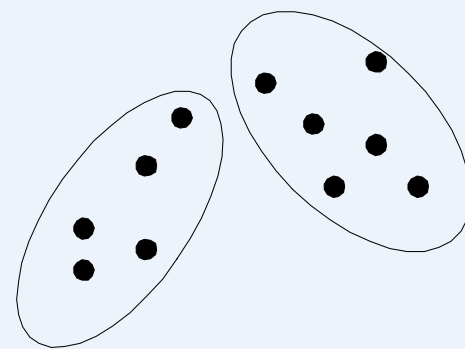
- A **clustering** is a set of clusters
- Important distinction between **hierarchical** and **partitional** sets of clusters
- Partitional Clustering (分割式聚类)
 - A division data objects into non-overlapping subsets (clusters) such that each data object is in exactly one subset 将数据对象划分为不重叠的子集（簇），使得每个数据对象恰好属于一个子集
 - # clusters is needed, kmeans....
- Hierarchical clustering (阶层式聚类)
 - A set of nested clusters organized as a hierarchical tree 一组嵌套簇，组织为层次树结构
 - # clusters is not needed



分割式聚类



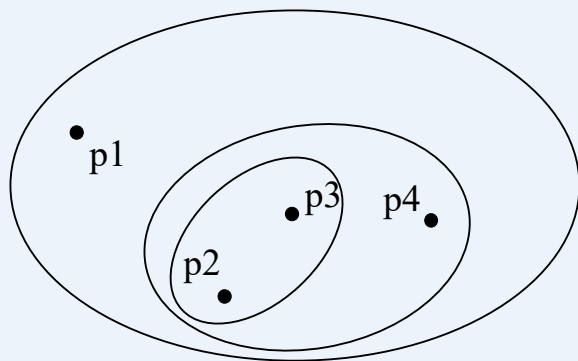
Original Points



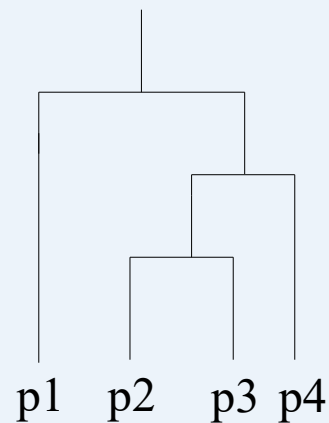
A Partitional Clustering



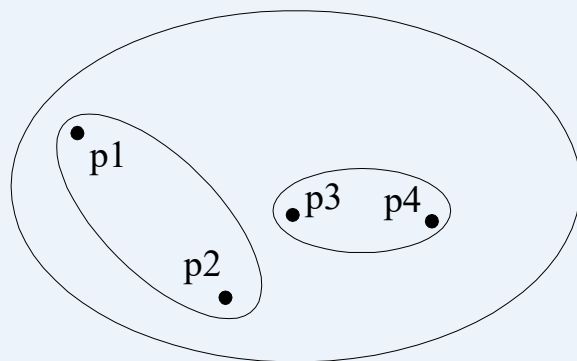
阶层式聚类



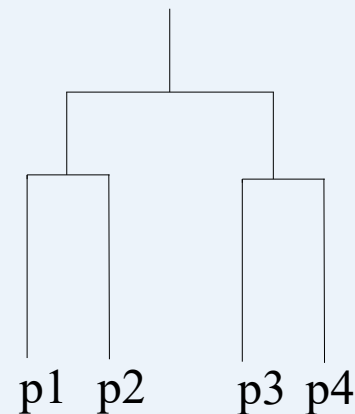
Traditional Hierarchical Clustering



Traditional Dendrogram



Non-traditional Hierarchical Clustering



Non-traditional Dendrogram



不同聚类集合的其他差异

- Fuzzy versus non-fuzzy 模糊聚类
 - In fuzzy clustering, a point belongs to every cluster with some weight between 0 and 1
 - Weights must sum to 1
 - Probabilistic clustering has similar characteristics



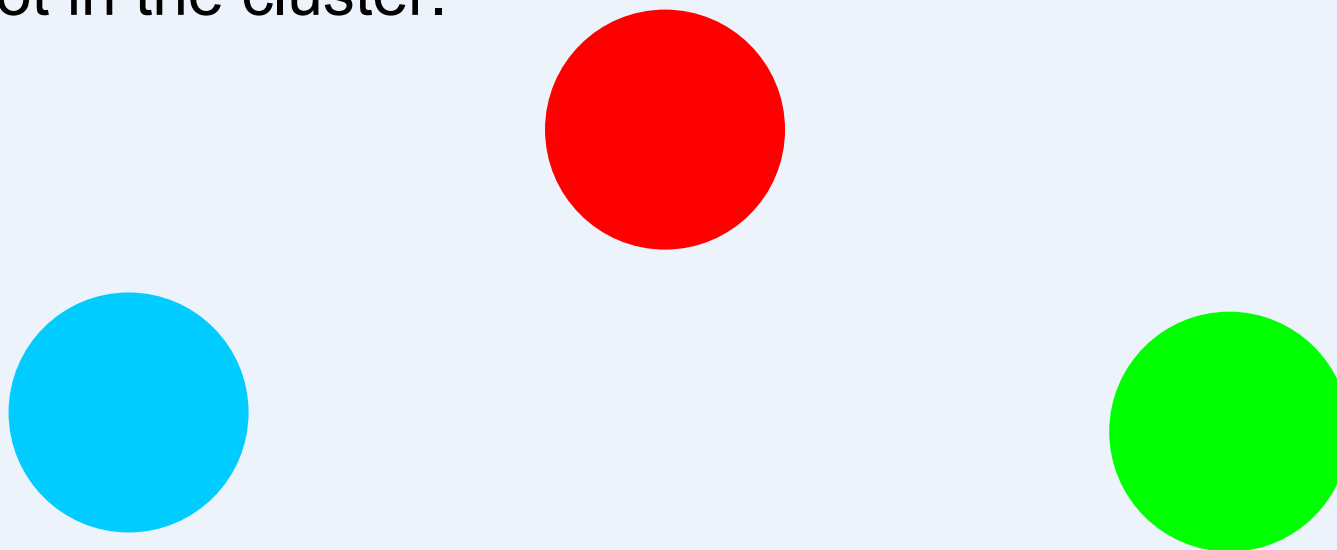
簇(Cluster)的类型

- Well-separated clusters 分离良好
- Center-based clusters 基于中心
- Contiguous clusters 基于邻近程度
- Density-based clusters 基于密度
- Property or Conceptual 基于属性或概念
- Described by an Objective Function 基于目标函数



Well-Separated Clusters

- Well-Separated Clusters:
 - A cluster is a set of points such that any point in a cluster is closer (or more similar) to every other point in the cluster than to any point not in the cluster.



3 well-separated clusters



Center-Based Clusters

- Center-based
 - A cluster is a set of objects such that an object in a cluster is closer (more similar) to the “center” of a cluster, than to the center of any other cluster
 - The center of a cluster is often a **centroid**, the average of all the points in the cluster, or a **medoid**, the most “representative” point of a cluster

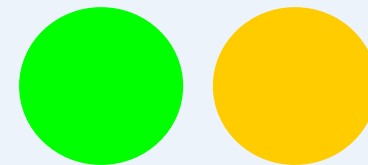
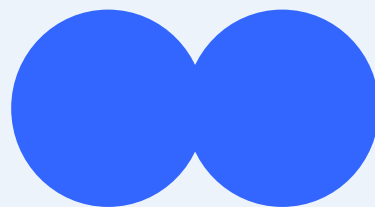
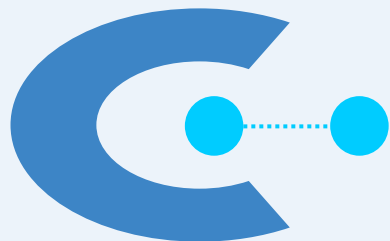
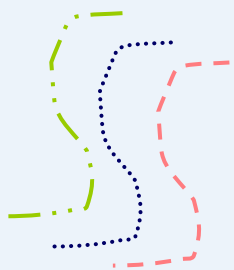


4 center-based clusters



Contiguity(邻近)-Based Clusters

- Contiguous Cluster (Nearest neighbor or Transitive)
 - A cluster is a set of points such that a point in a cluster is closer (or more similar) to one or more other points in the cluster than to any point not in the cluster.

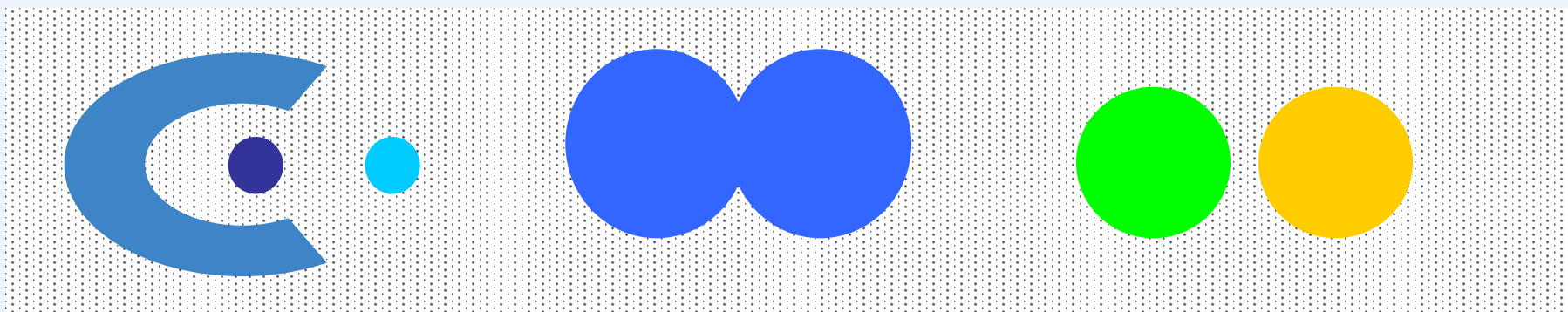


8 contiguous clusters



Density-Based Clusters

- Density-based
 - A cluster is a dense region of points, which is separated by low-density regions, from other regions of high density.
 - Used when the clusters are irregular or intertwined(不规则or纠缠), and when noise and outliers are present.

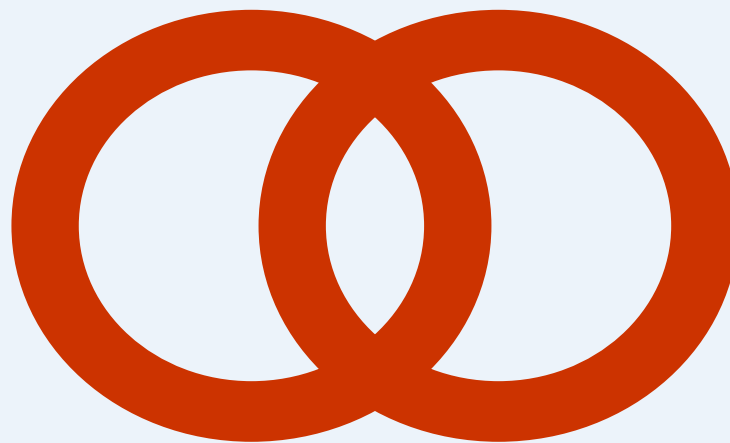


6 density-based clusters



Conceptual(概念) Clusters

- Shared Property or Conceptual Clusters
 - Finds clusters that share some common property or represent a particular concept.



2 Overlapping Circles



Objective Function

- 由目标函数定义的聚类
- Finds clusters that minimize or maximize an objective function. 寻找使目标函数最小化或最大化的聚类。
 - **穷举法:** Enumerate all possible ways of dividing the points into clusters and evaluate the 'goodness' of each potential set of clusters by using the given objective function. 枚举将数据点划分为聚类的所有可能方式，并使用给定的目标函数评估每个潜在聚类集合的“优度” (NP Hard)
 - Can have global or local objectives. 可以具有全局或局部目标



Objective Function ...

- Map the clustering problem to a different domain and solve a related problem in that domain
 - Proximity (亲近度) matrix defines a weighted graph, where the nodes are the data points, and the weighted edges represent the proximities (similarity or dissimilarity) between data points

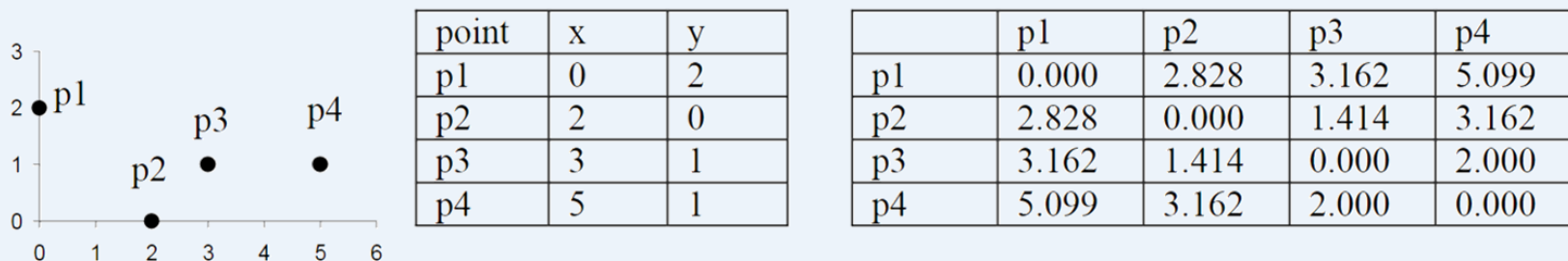


Figure Four points and their corresponding data and proximity (distance) matrices.

- Similarity? Dissimilarity?

Euclidean Distance / Manhattan Distance / Chebyshev Distance /

Cosine Similarity /

$$\text{Similarity} = 1 - \text{Dissimilarity}$$



距离度量指标

- $L_p(x_1, x_2) = \left(\sum_{l=1}^n |x_i^{(l)} - x_j^{(l)}|^p \right)^{\frac{1}{p}}$
- $L_1(x_1, x_2) = \sum_{l=1}^n |x_i^{(l)} - x_j^{(l)}|$ Manhattan Distance
- $L_2(x_1, x_2) = \sum_{l=1}^n \sqrt{(x_i^{(l)} - x_j^{(l)})^2}$ Euclidean Distance
- $L_\infty(x_1, x_2) = \sqrt[n]{\sum_{l=1}^n |x_i^{(l)} - x_j^{(l)}|^\infty} = \max(|x_i - x_j|)$
Chebyshev Distance
- $L_{-\infty}(x_1, x_2) = \sqrt[-\infty]{\sum_{l=1}^n |x_i^{(l)} - x_j^{(l)}|^{-\infty}} = \min(|x_i - x_j|)$
- Cosine Distance: $D(x_1, x_2) = 1 - \cos(x_1, x_2) = 1 - \frac{x_1 \cdot x_2}{\|x_1\|_2 \|x_2\|_2}$

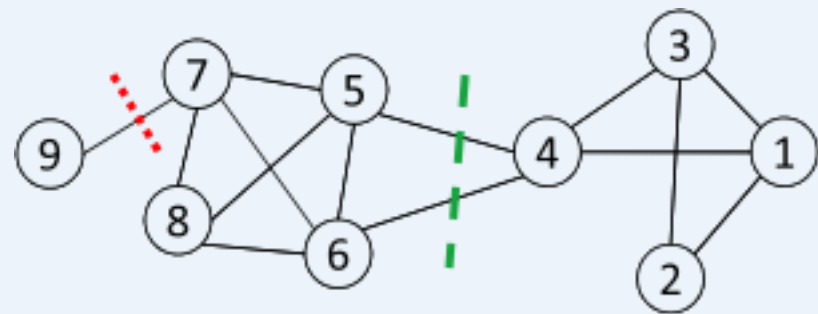


Objective Function ...

- Clustering is equivalent to breaking the graph into connected components, one for each cluster.
- Want to minimize the edge weight between clusters and maximize the edge weight within clusters

$$\text{Ratio Cut}(\pi) = \frac{1}{k} \sum_{i=1}^k \frac{\text{cut}(C_i, \bar{C}_i)}{|C_i|},$$

$$\text{Normalized Cut}(\pi) = \frac{1}{k} \sum_{i=1}^k \frac{\text{cut}(C_i, \bar{C}_i)}{\text{vol}(C_i)}$$



C_i : a community

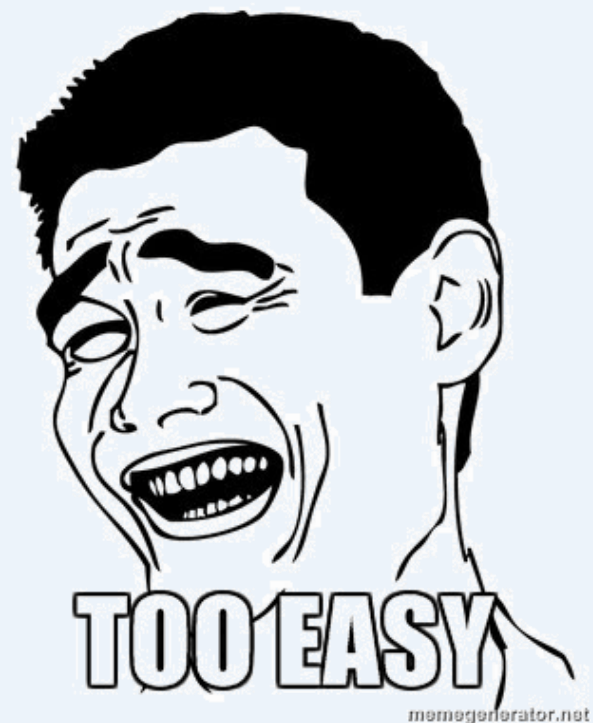
$|C_i|$: # nodes in C_i

$\text{vol}(C_i)$: sum of degrees in C_i



聚类算法

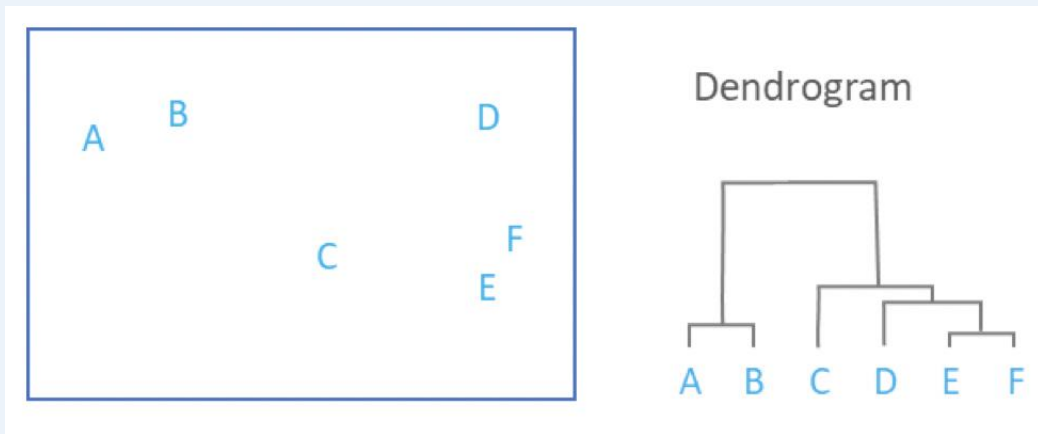
- Hierarchical clustering (Omit, because...)
- K-means and its variants
- Density-based clustering





阶层式聚类算法

● Hierarchical clustering

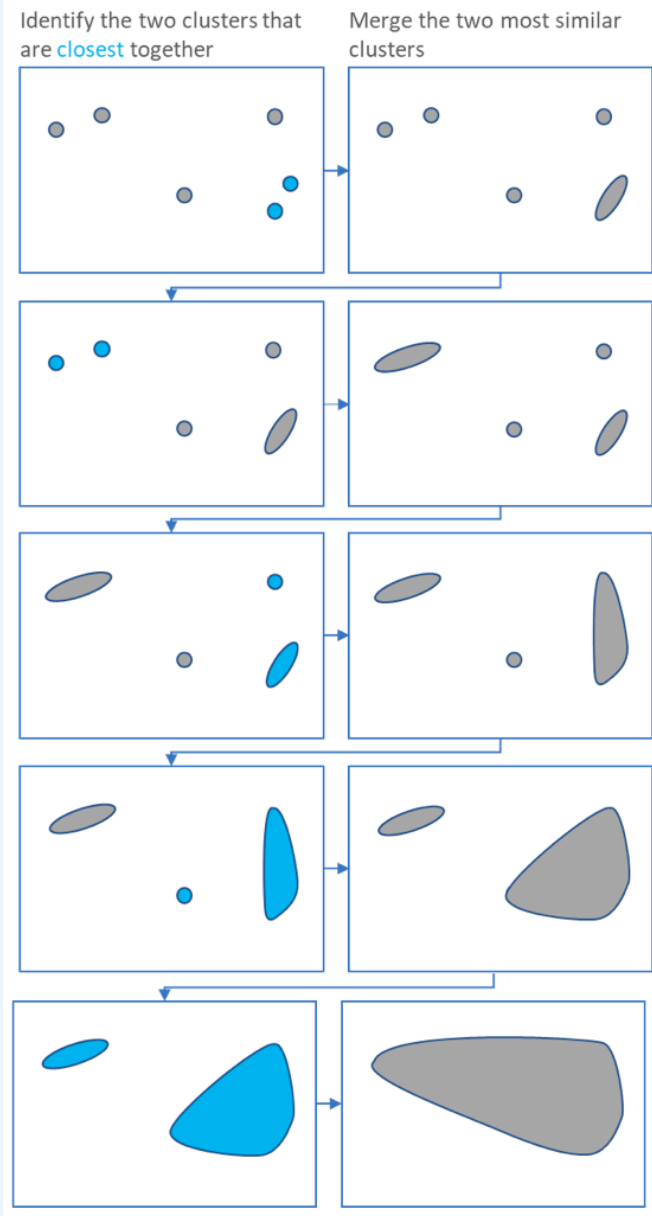


层次聚类从将每个观测值视为一个独立簇开始。

随后，它循环执行以下两个步骤：

- ① 识别距离最近的两个簇
- ② 合并最相似的两个簇

此迭代过程持续进行，直到所有簇合并为一个整体。



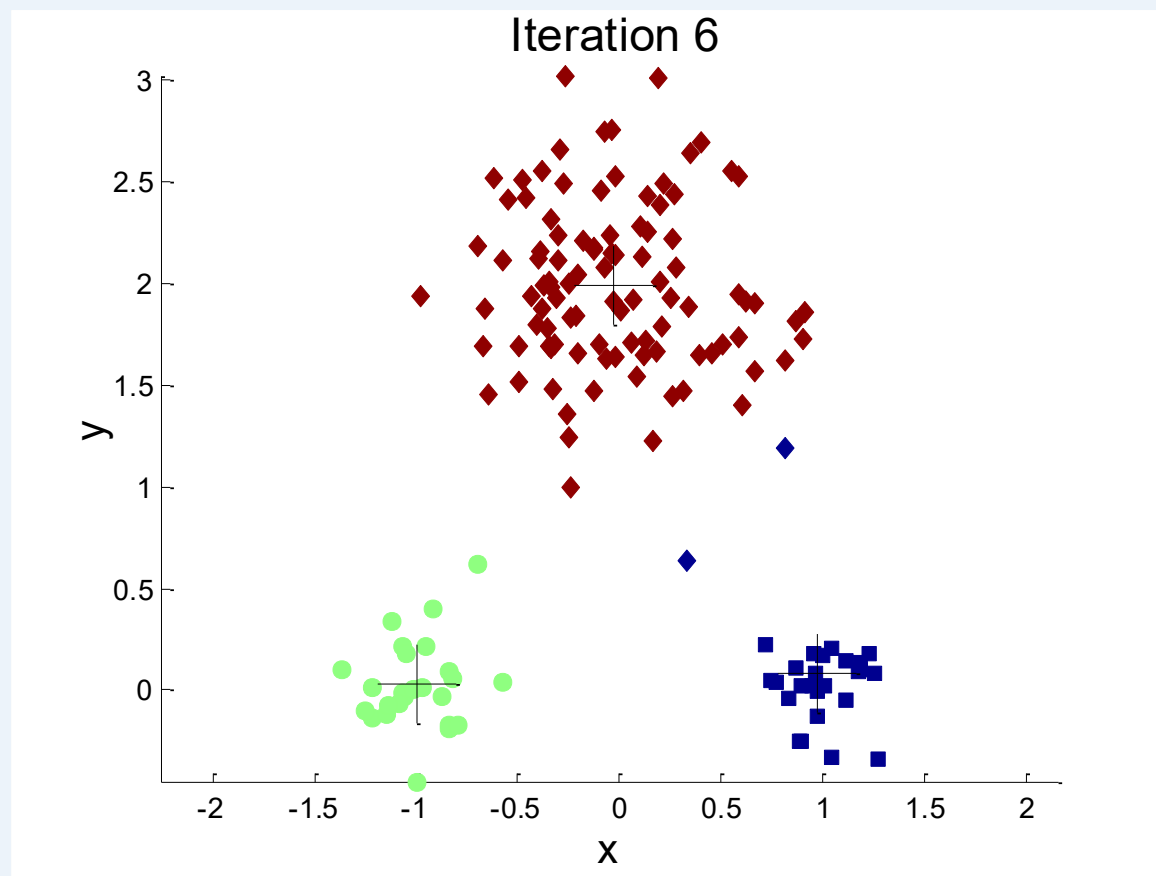
K-means聚类

- Partitional clustering approach 分割式聚类方法
 - Each cluster is associated with a **centroid** (center point) 每个簇有一个中心点
 - Each point is assigned to the cluster with the **closest centroid** 将各数据点划分到最近中心点对应的簇
 - Number of clusters, K , must be specified
 - The basic algorithm is very simple

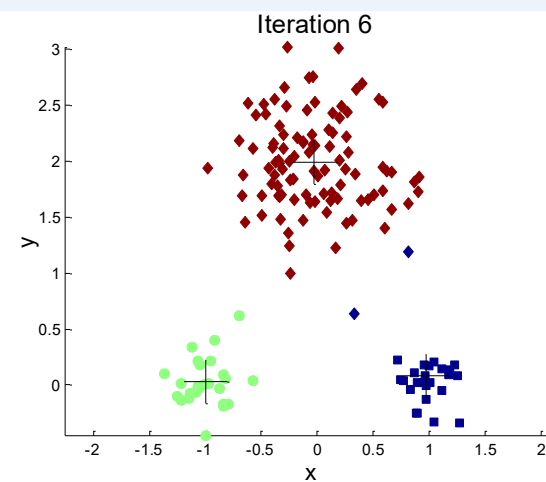
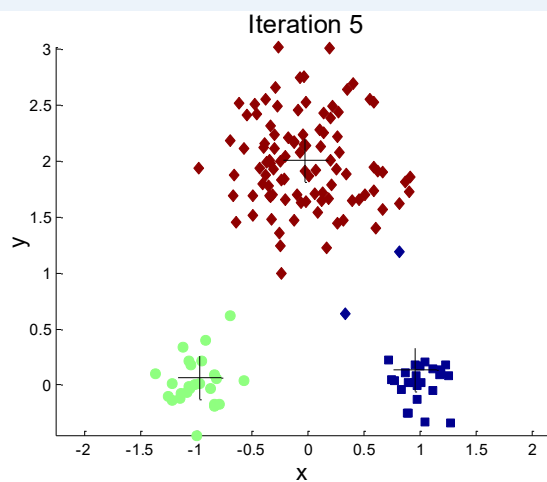
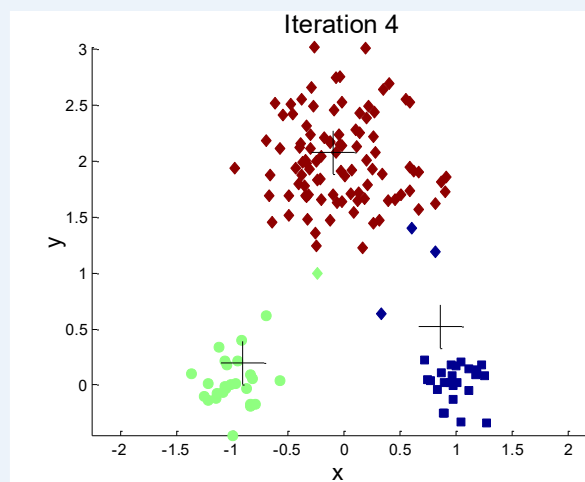
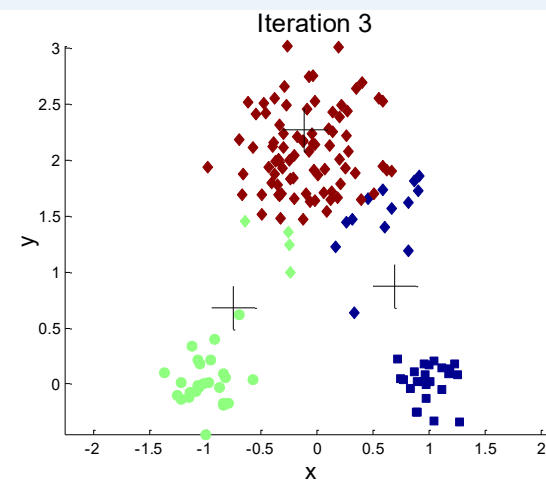
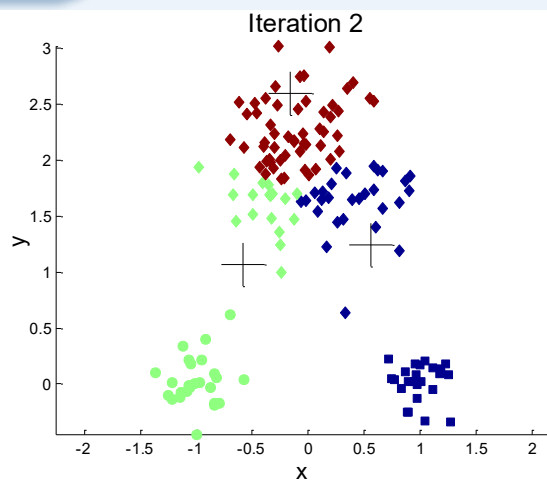
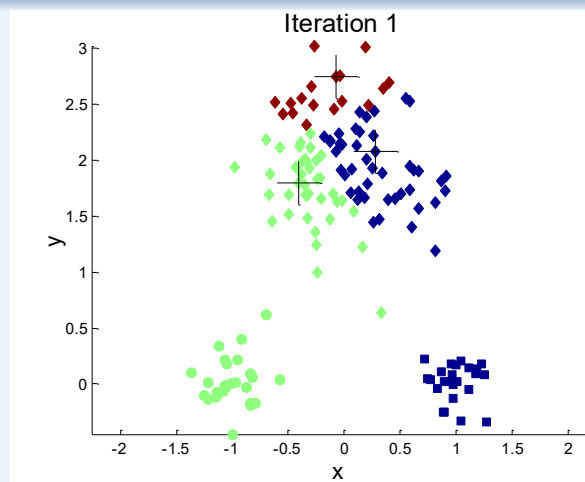
算法 1 基础 K-means 算法。

- 1: 选择 K 个点作为初始中心。
- 2: **重复**
- 3: 通过将所有点分配给最近的中心来形成 K 个簇。
- 4: 重新计算每个簇的中心。
- 5: **直到** 中心不再发生变化

标准K-means流程



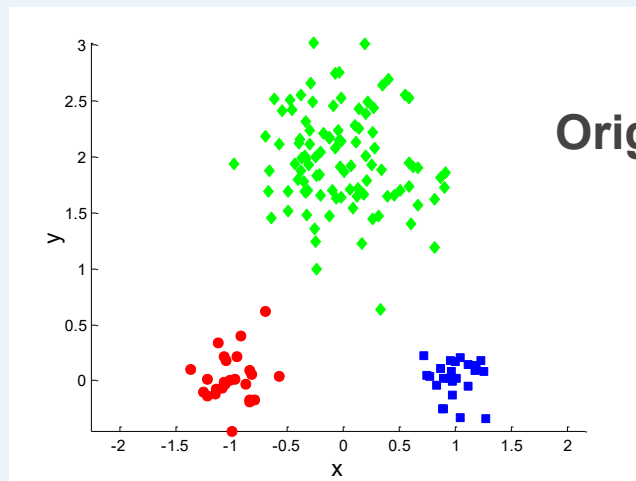
标准K-means流程



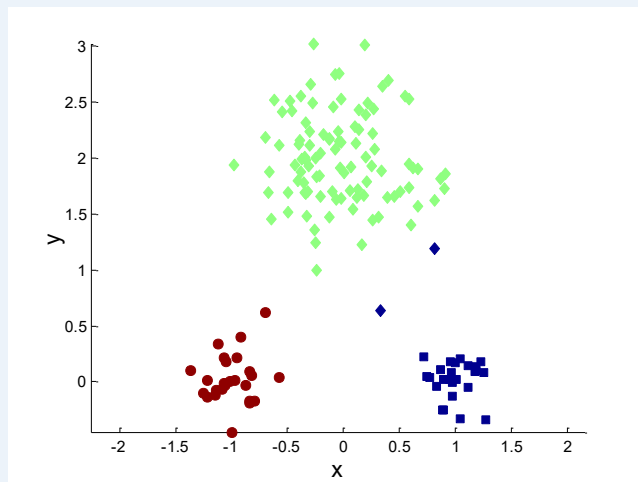
K-mean聚类细节

- 初始中心点通常随机选择。
 - ✓ 不同次运行得到的聚类结果可能不同。
- 中心点通常是簇内各点的均值。
- “相近度”可通过欧氏距离、余弦相似度、相关系数等度量。
- 使用上述常见相似性度量时，K-means算法能够保证收敛。
- 大部分收敛发生在最初几次迭代中。Often the stopping condition is changed to 'Until relatively few points change clusters'
- Complexity is $O(n * K * I * d)$
 - n = number of points, K = number of clusters,
 I = number of iterations, d = number of attributes

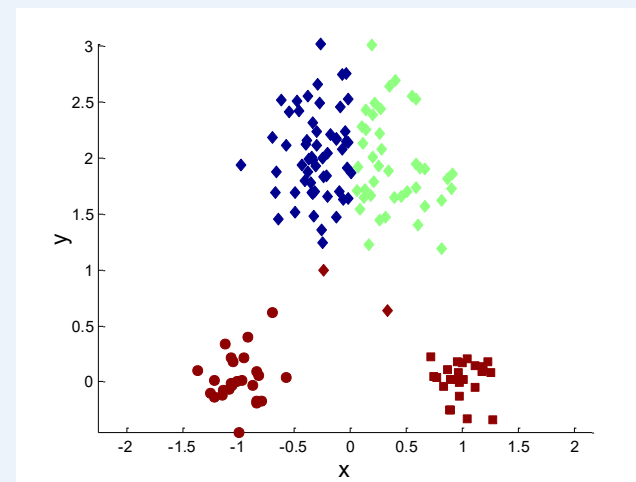
两个不同的K-mean聚类



Original Points

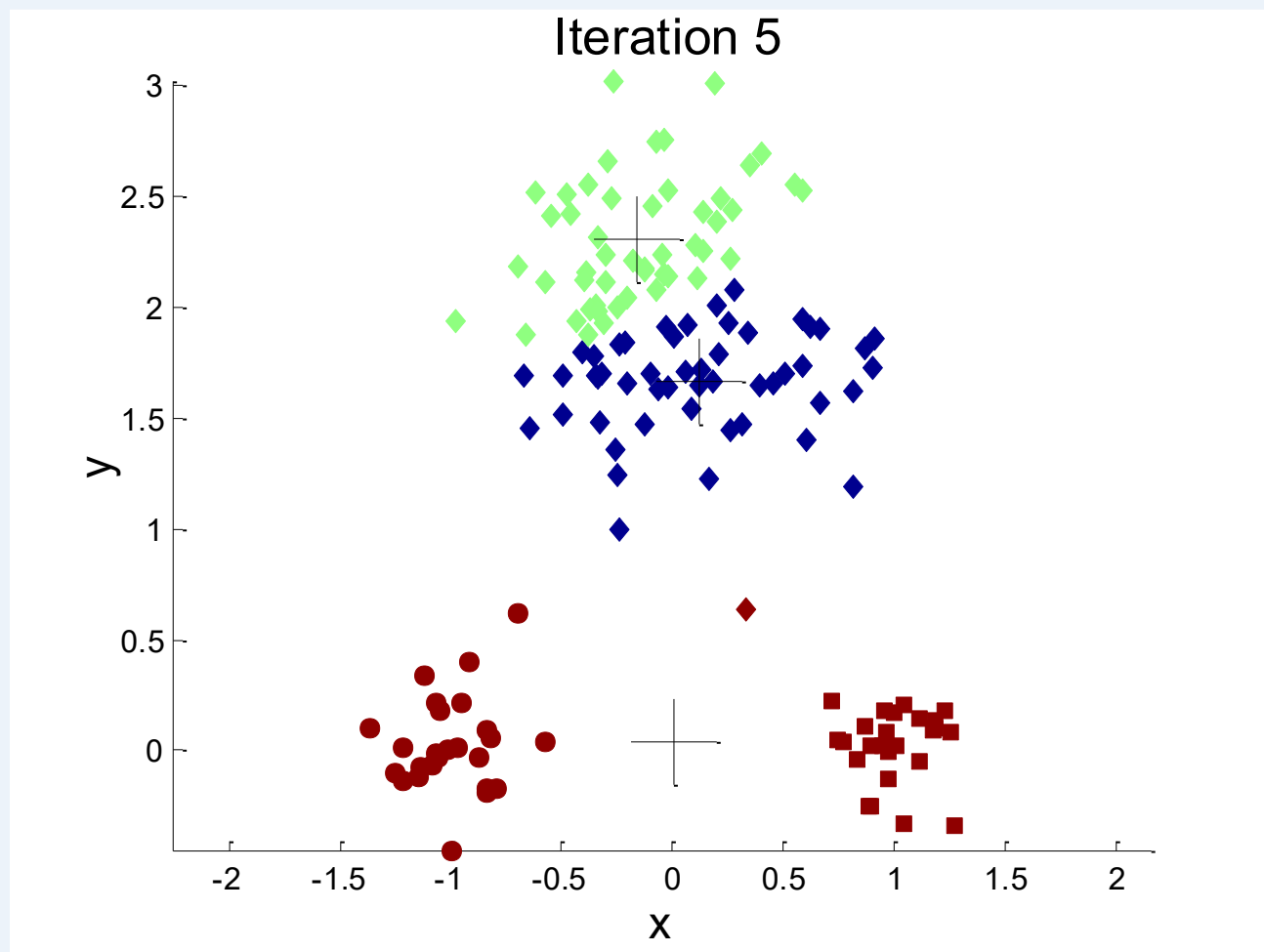


Optimal Clustering

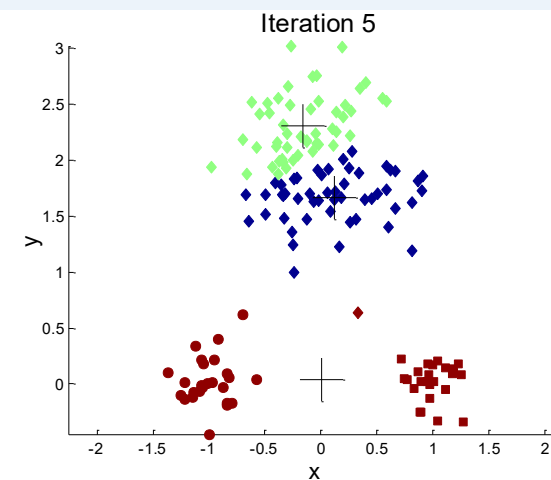
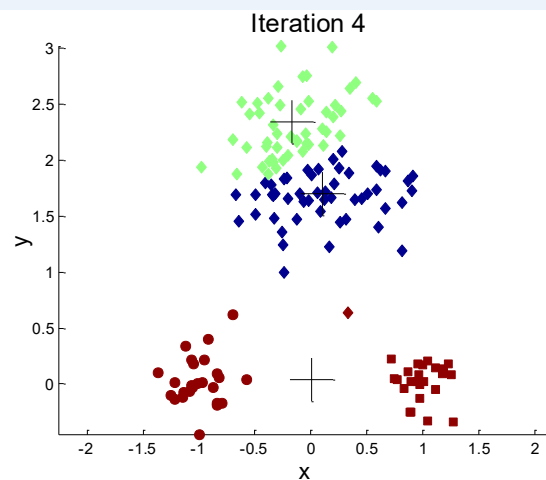
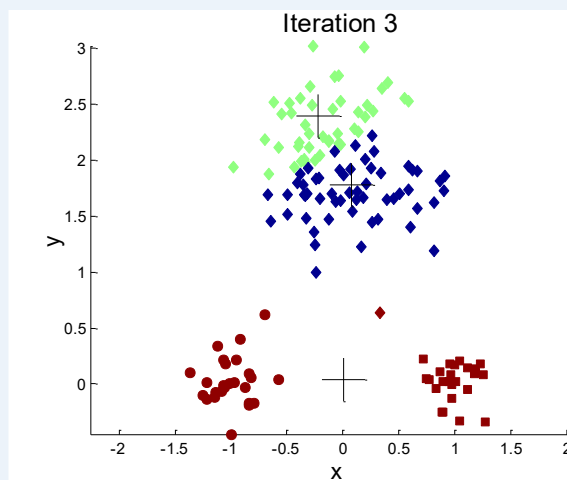
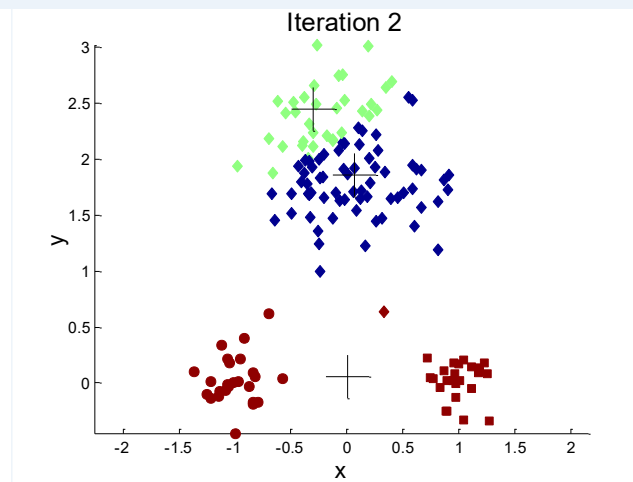
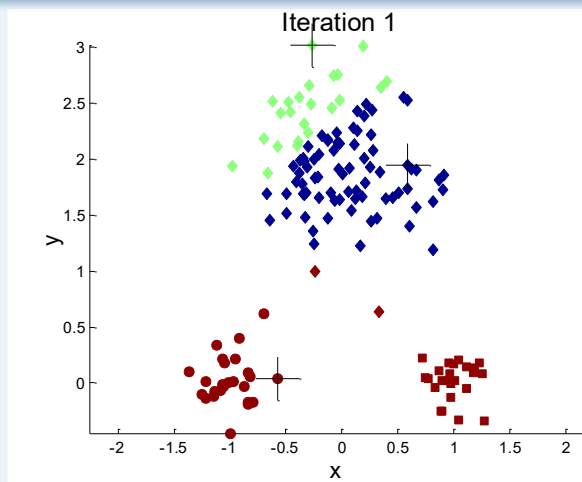


Sub-optimal Clustering

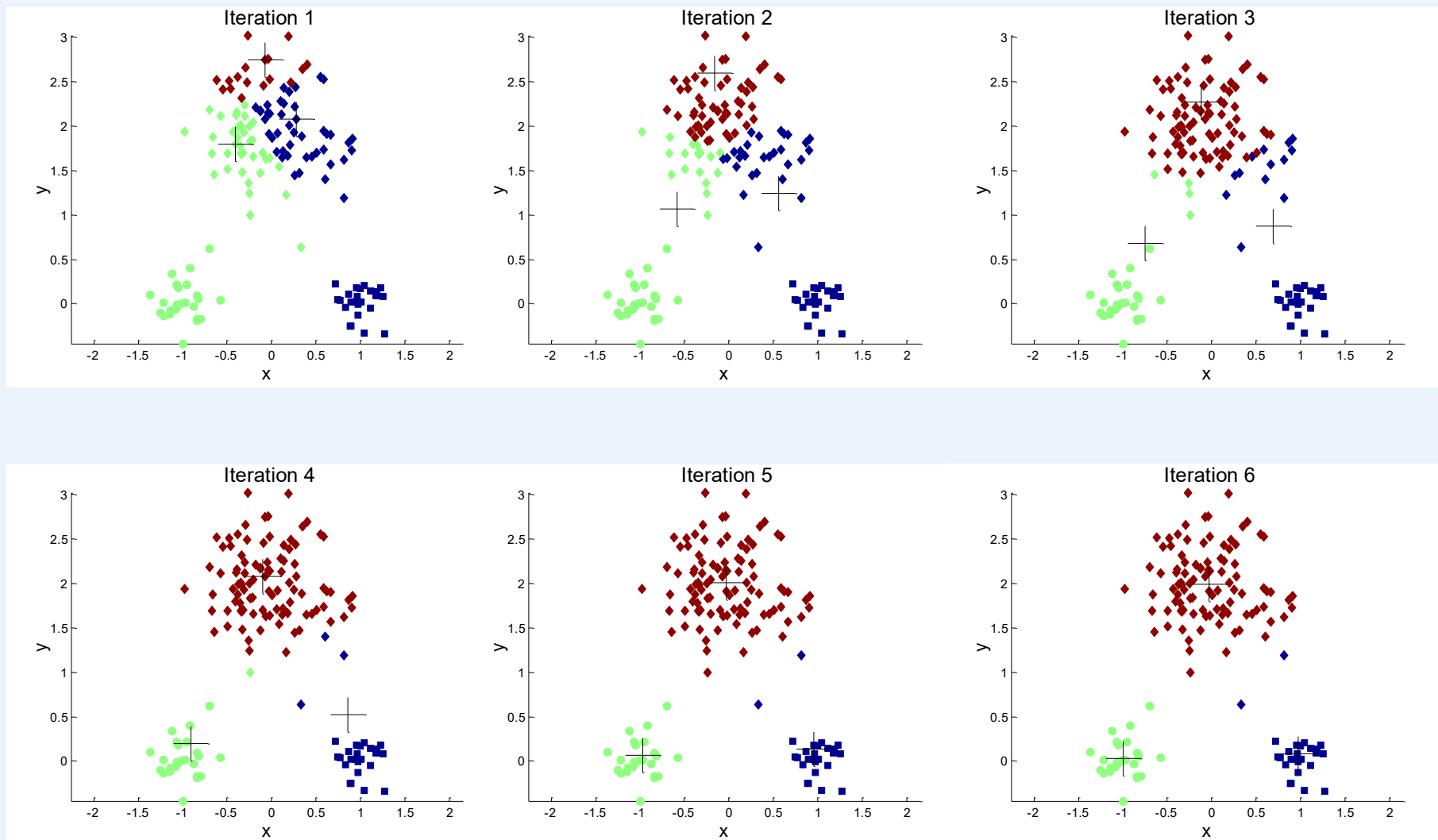
选择初始质心的重要性



选择初始质心的重要性



选择初始质心的重要性



K-means簇的评估

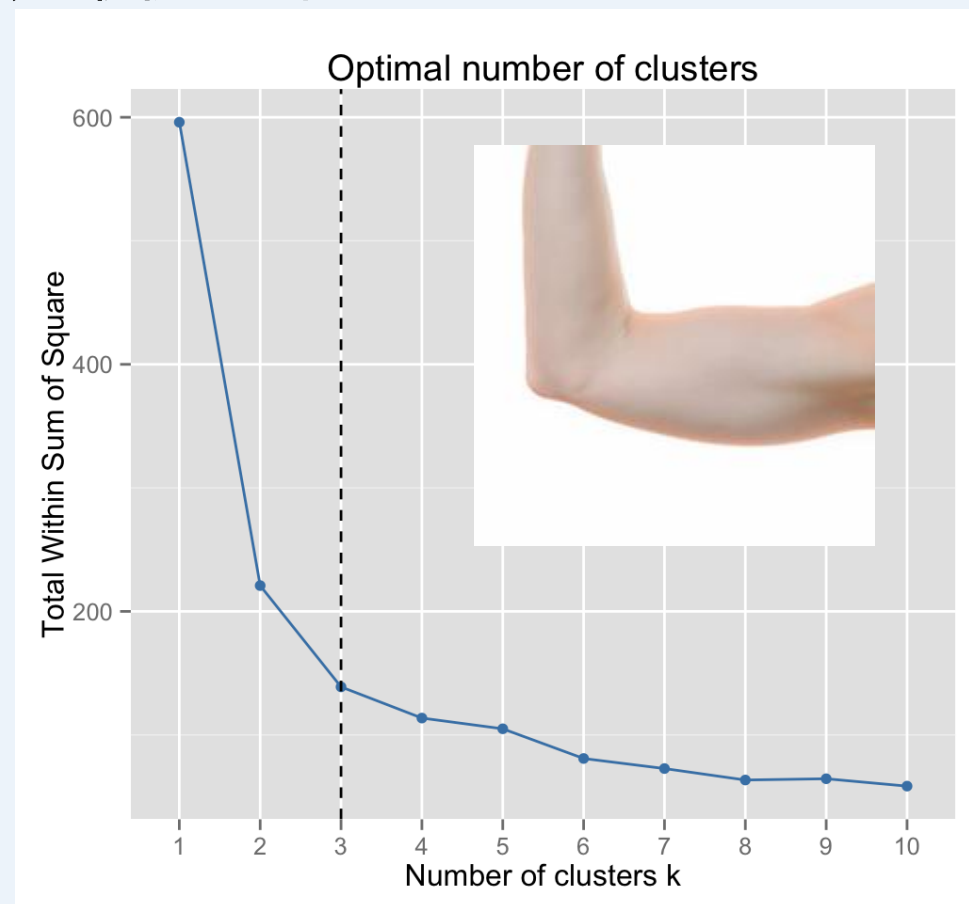
- 最常用的度量是误差平方和Sum of Squared Error (SSE)
 - 计算每个点到其最近聚类中心的距离作为误差
 - 将所有这些误差平方后求和，即得到SSE

$$SSE = \sum_{i=1}^K \sum_{x \in C_i} dist^2(m_i, x)$$

- x 表示簇 C_i 中的数据， m_i 代表簇 C_i 中心点
- 给定两个聚类方案时，我们选择误差更小的那个
- 降低SSE最直接的方法是增加聚类数量K
 - 一个K值较小的优质聚类，其SSE可能低于K值更大的劣质聚类

K的选择

- **Elbow method肘部法则**：绘制不同K值对应的SSE折线图，若图形呈手臂状，则"肘关节"处对应的K值即为最优选择。
- 手臂曲线的肘关节



<https://bl.ocks.org/rpgove/0060ff3b656618e9136b>

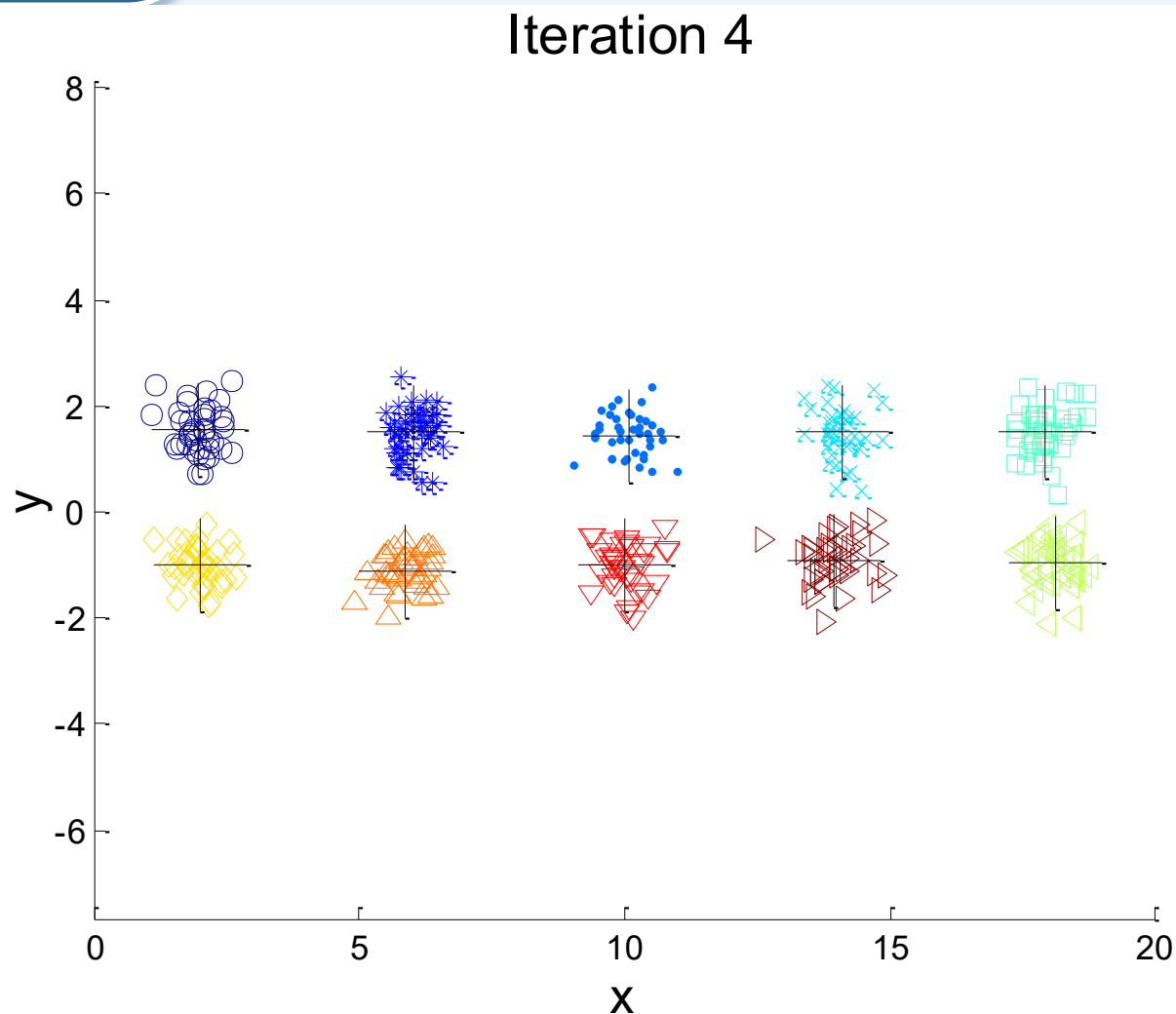
选择初始点的问题

- 若存在K个真实簇群，则从每个簇中各选出一个中心点的概率很小。
 - 当K值较大时，这种概率会显著降低
 - 假设各簇规模相同（均为n个数据点），则：

$$P = \frac{\text{number of ways to select one centroid from each cluster}}{\text{number of ways to select } K \text{ centroids}} = \frac{K!n^K}{(Kn)^K} = \frac{K!}{K^K}$$

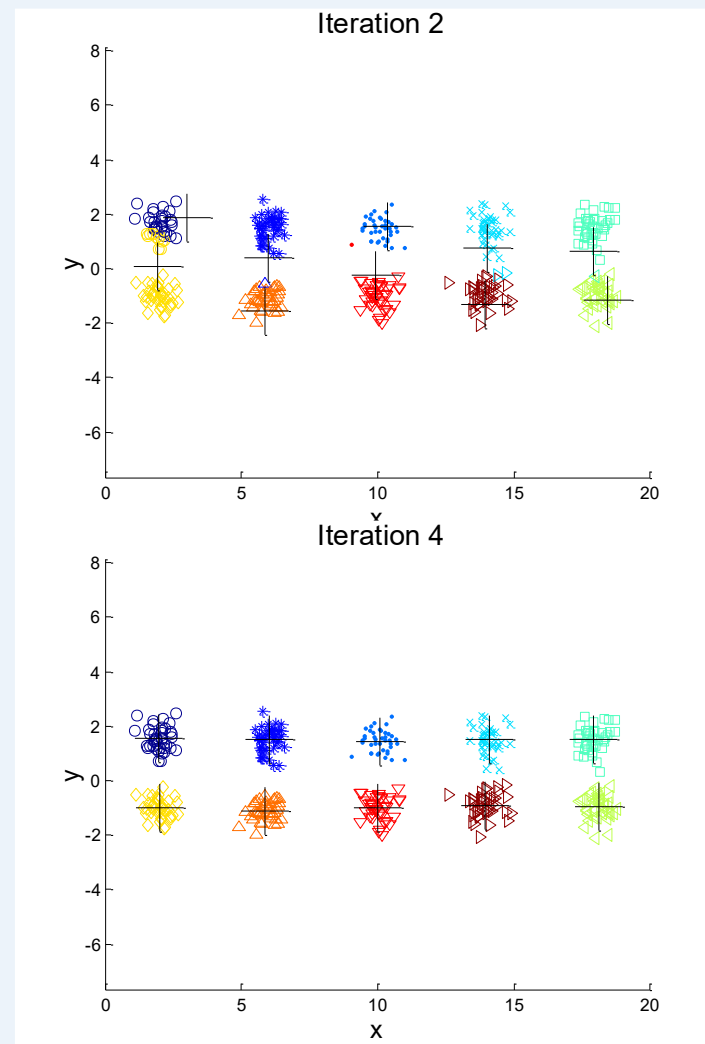
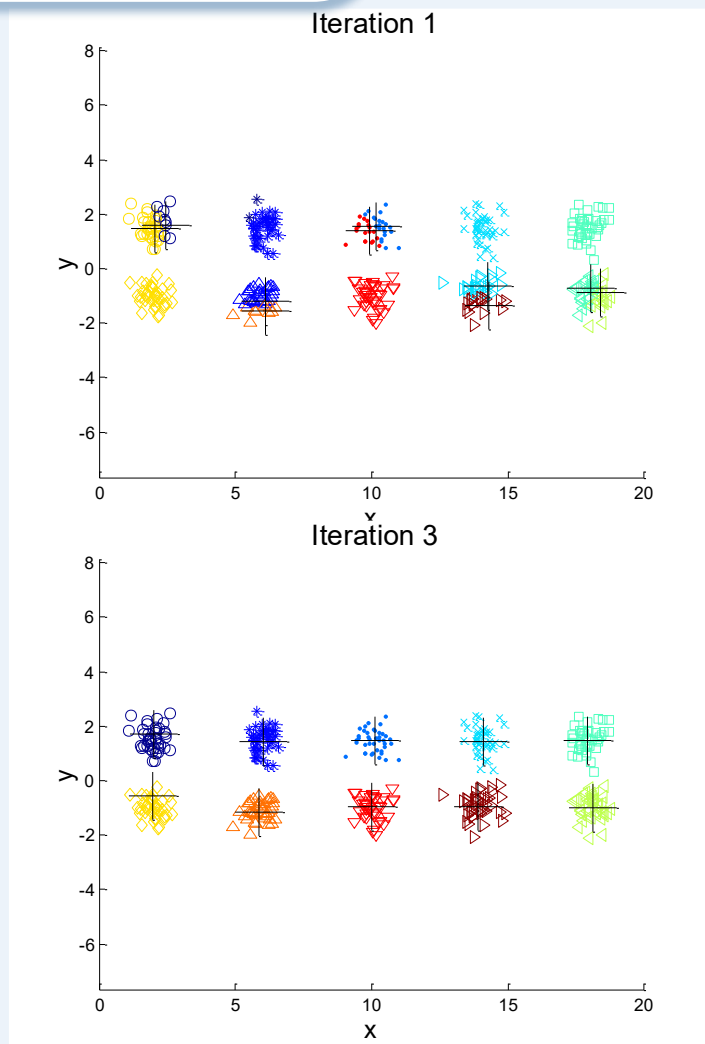
- For example, if $K = 10$, then probability = $10!/10^{10} = 0.00036$
- 初始中心点有时会自我调整到正确位置，有时则不能
- 以10个簇群的情况为例

例子：10个簇



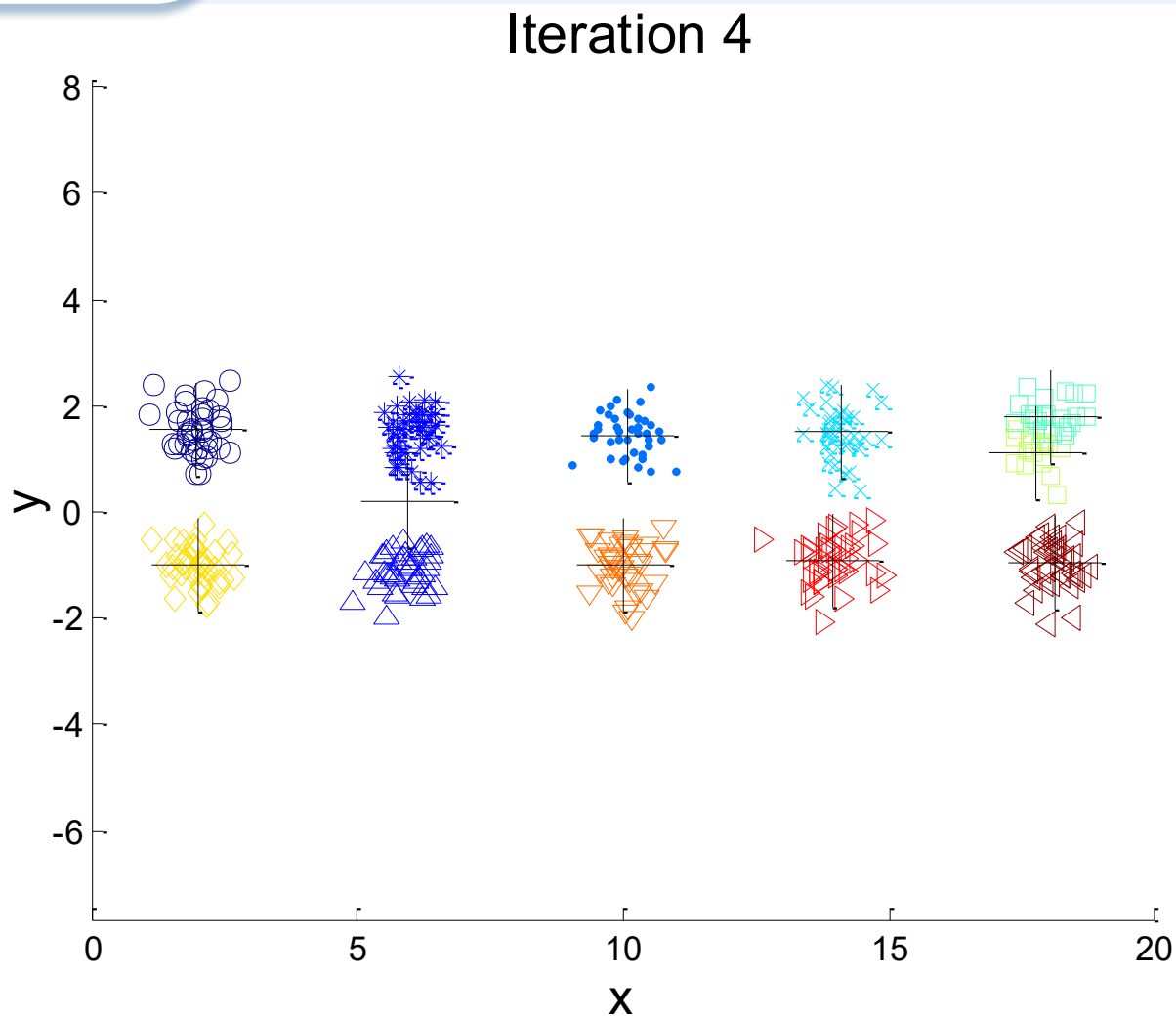
Starting with two initial centroids in one cluster of each pair of clusters

例子：10个簇



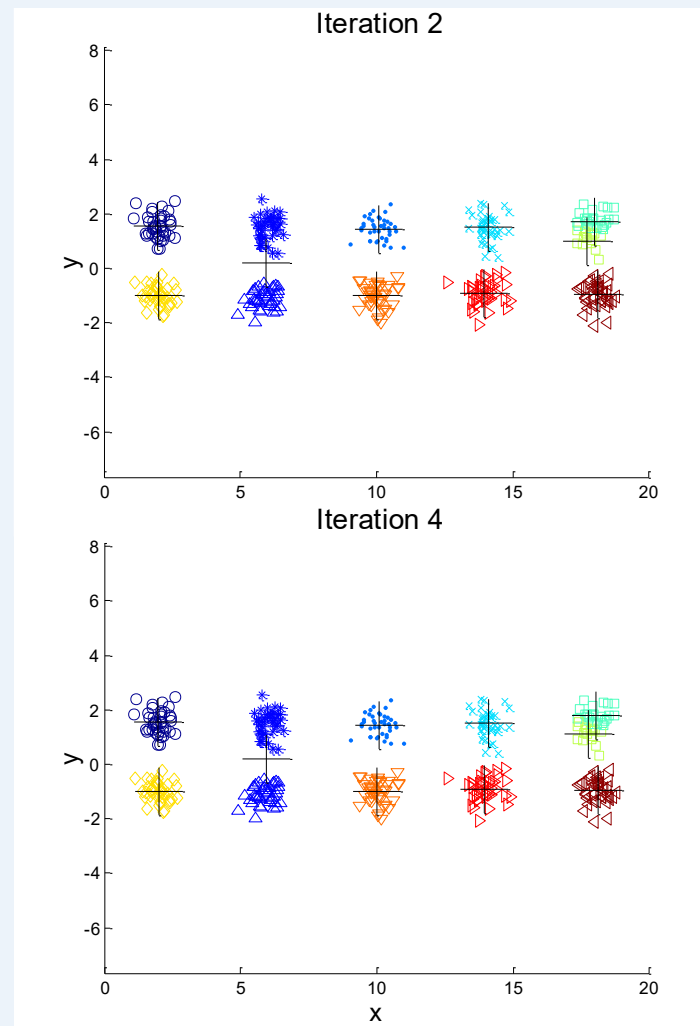
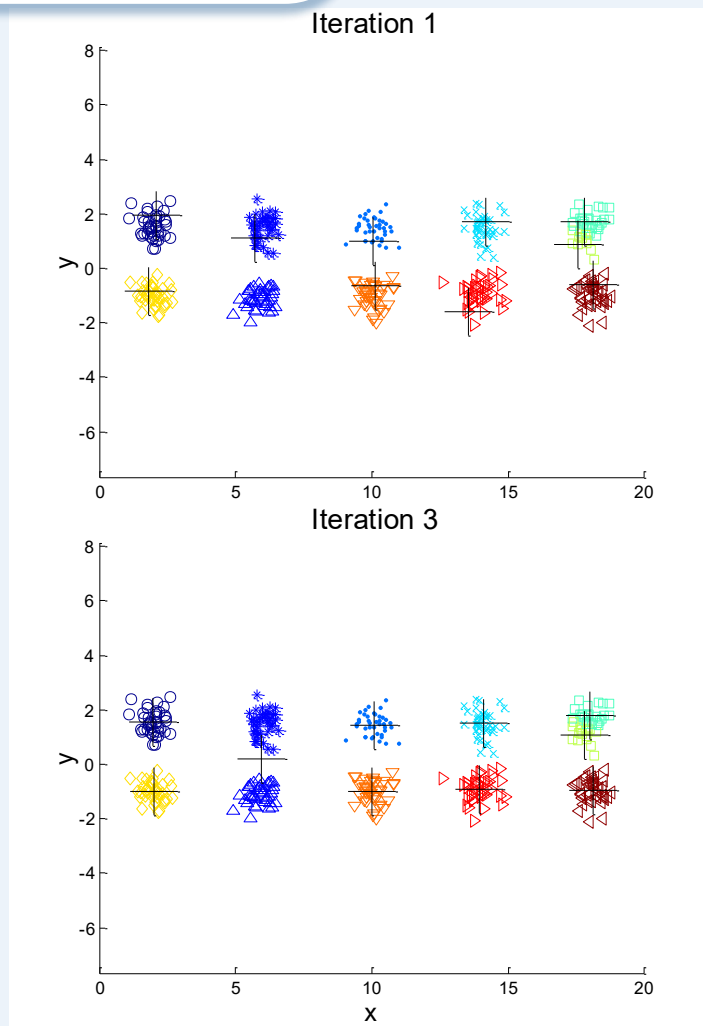
Starting with two initial centroids in one cluster of each pair of clusters

例子：10个簇



Starting with some pairs of clusters having three initial centroids, while other have only one.

例子：10个簇



Starting with some pairs of clusters having three initial centroids, while other have only one.

如何选择初始质心？

- 多次运行
 - ✓ 虽有效果，但成功概率依然有限
- 二分K均值法
 - ✓ 采用分裂式聚类策略：每次选择一个簇进行二分，最终形成K个簇
- 采样结合层次聚类
 - ✓ 先对数据采样，再用层次聚类确定初始中心点
- 优选中心点
 - ✓ 选取多于K个初始中心点，从中筛选分布最分散的K个点

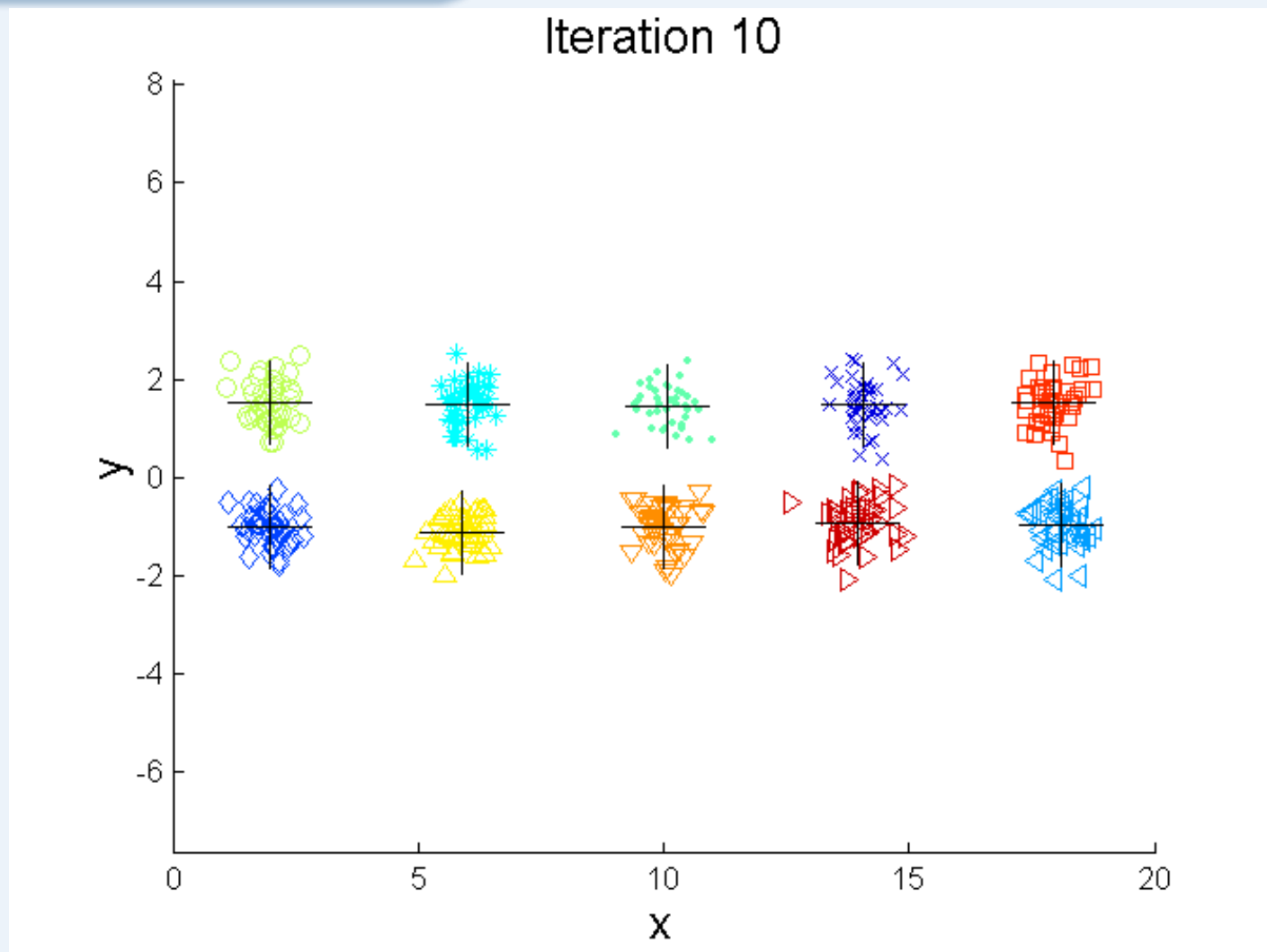
Bisecting(二分) K-means

- Bisecting K-means algorithm
 - Variant of K-means that can produce a partitional or a hierarchical clustering

算法 3 二分 K-means 算法。

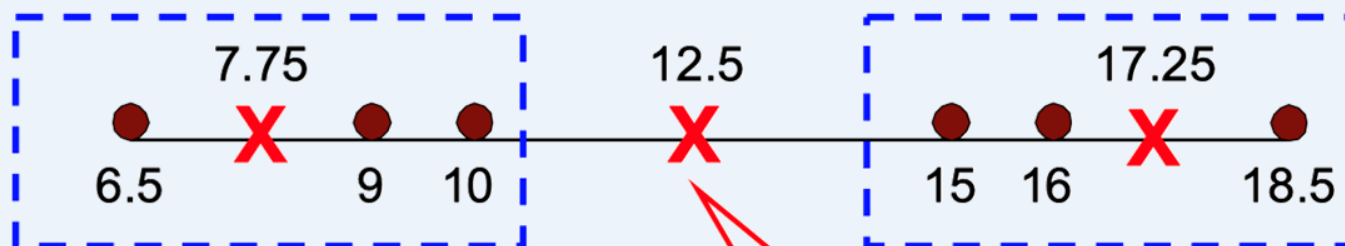
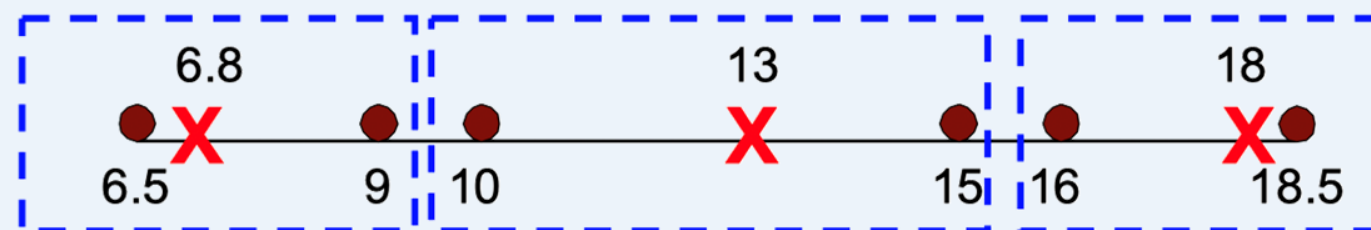
- 1: 初始化簇列表，使其包含涵盖所有点的簇。
- 2: **重复**
- 3: 从簇列表选择一个簇
- 4: **对于** $i = 1$ **到** $number_of_iterations$ **执行**
- 5: 使用基础 K-means 对所选簇进行二分
- 6: **结束 for**
- 7: 将具有最低 SSE 的二分结果中的两个簇添加到簇列表中。
- 8: **直到** 簇列表包含 K 个簇

例子：二分K-means



处理空簇

- Basic K-means algorithm can yield **empty clusters**



Empty
Cluster

处理空簇

- K均值算法可能产生空簇。
- 若出现这种情况，可采取以下策略选择替代中心点：
 - ✓ 选取对**SSE**贡献最大的数据点
 - ✓ 从具有最高**SSE**的簇中选择一个数据点
- 若存在多个空簇，可重复执行上述操作。

$$SSE = \sum_{i=1}^K \sum_{x \in C_i} dist^2(m_i, x)$$

预处理和后处理

预处理阶段

- ✓ 数据标准化
- ✓ 剔除异常值

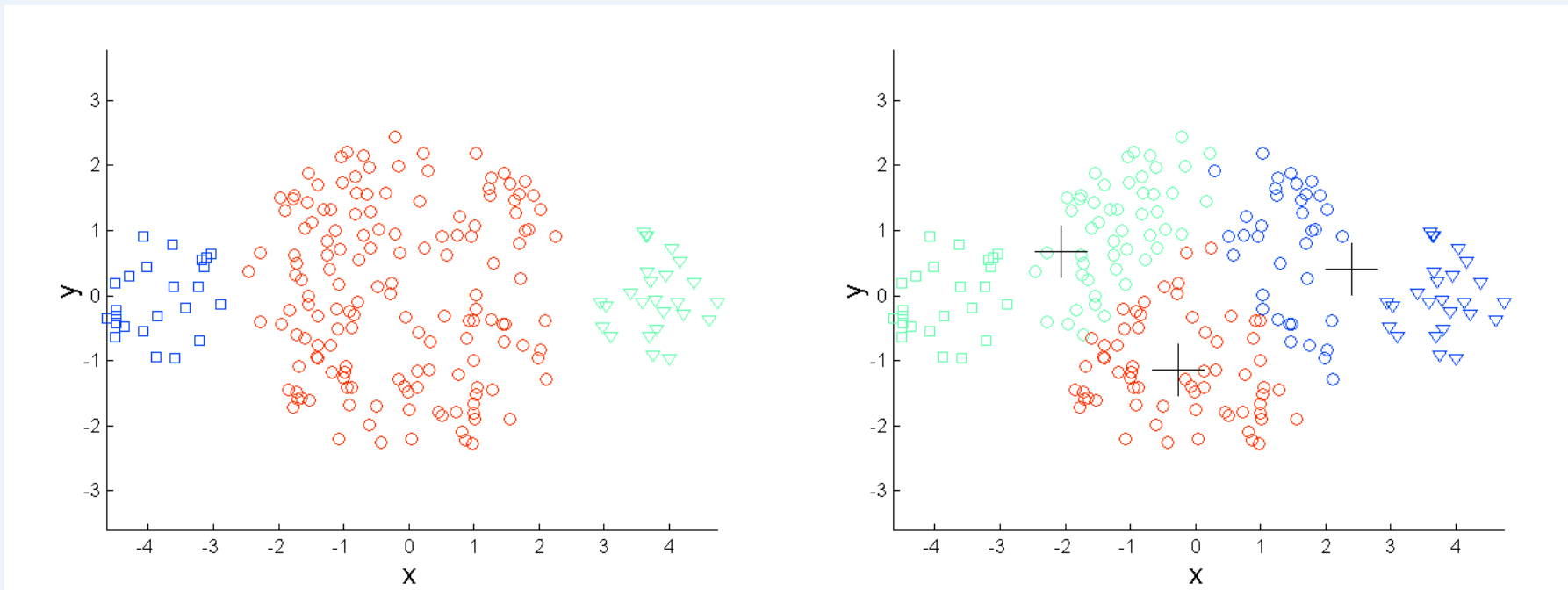
后处理阶段

- ✓ 剔除可能代表异常值的小规模簇
- ✓ 拆分结构松散的簇（即SSE较高的簇）
- ✓ 合并距离相近且SSE较低的簇

K-means的其他局限性

- K-means has problems when clusters are of differing
 - Sizes
 - Densities
 - Non-globular shapes
- K-means has problems when the data contains outliers.

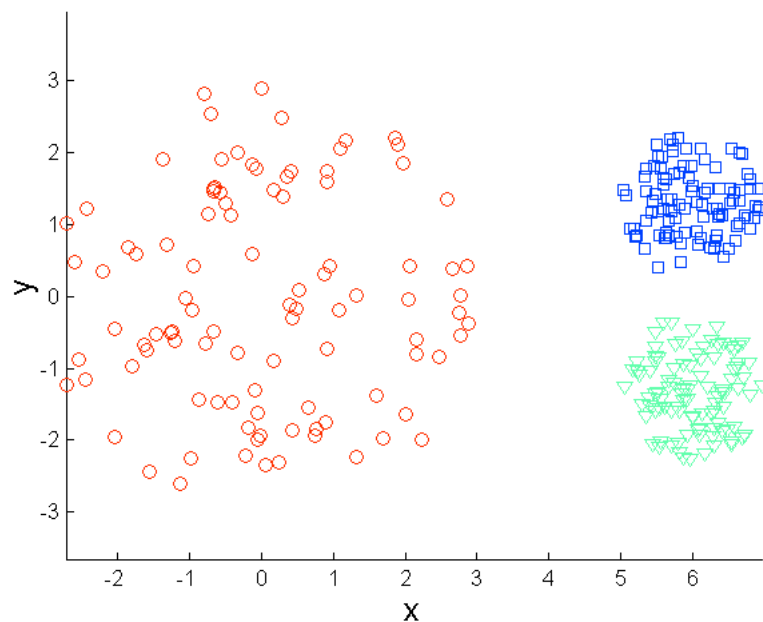
K-means的局限: 不同大小



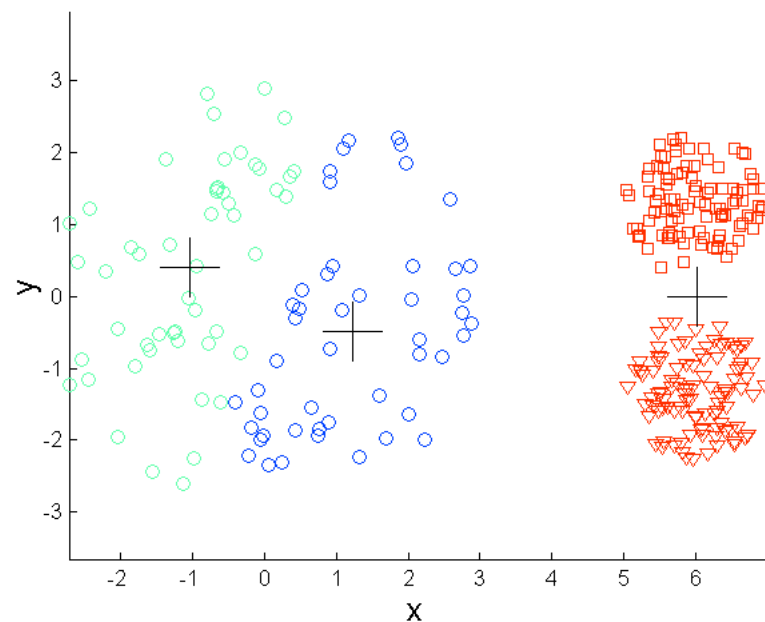
Original Points

K-means (3 Clusters)

K-means的局限: 不同密度

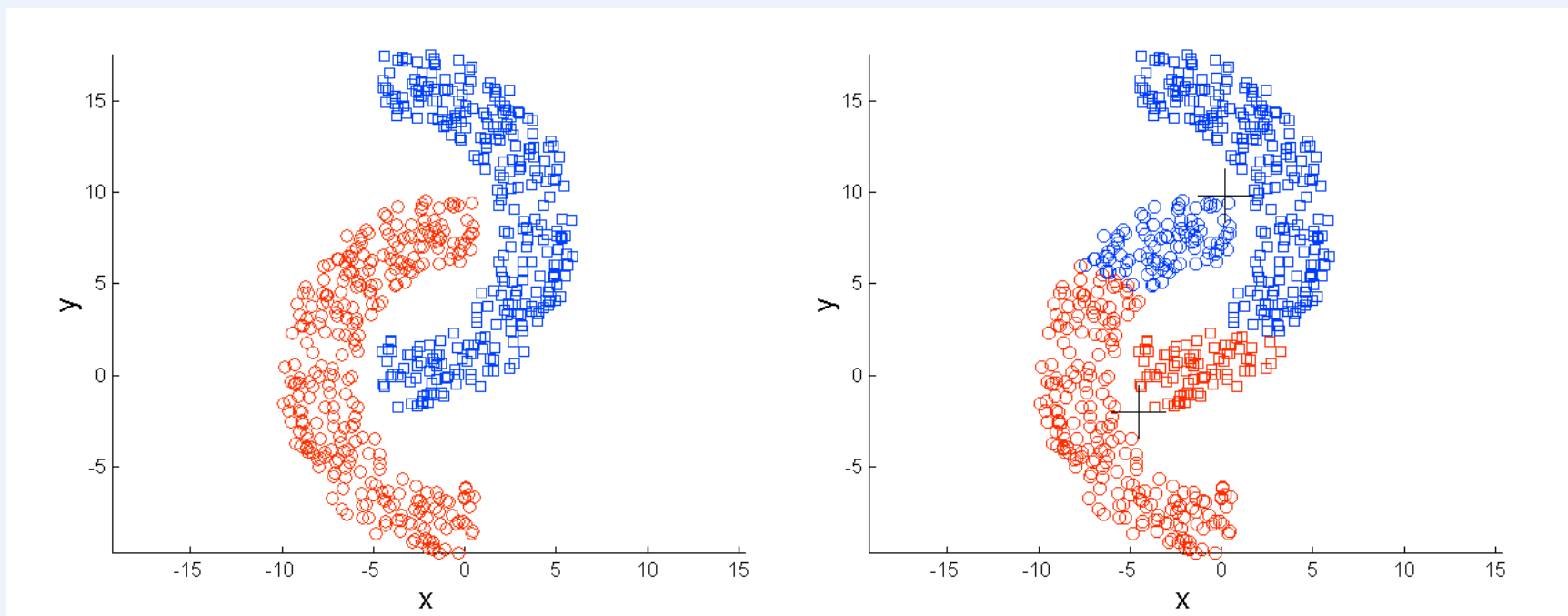


Original Points



K-means (3 Clusters)

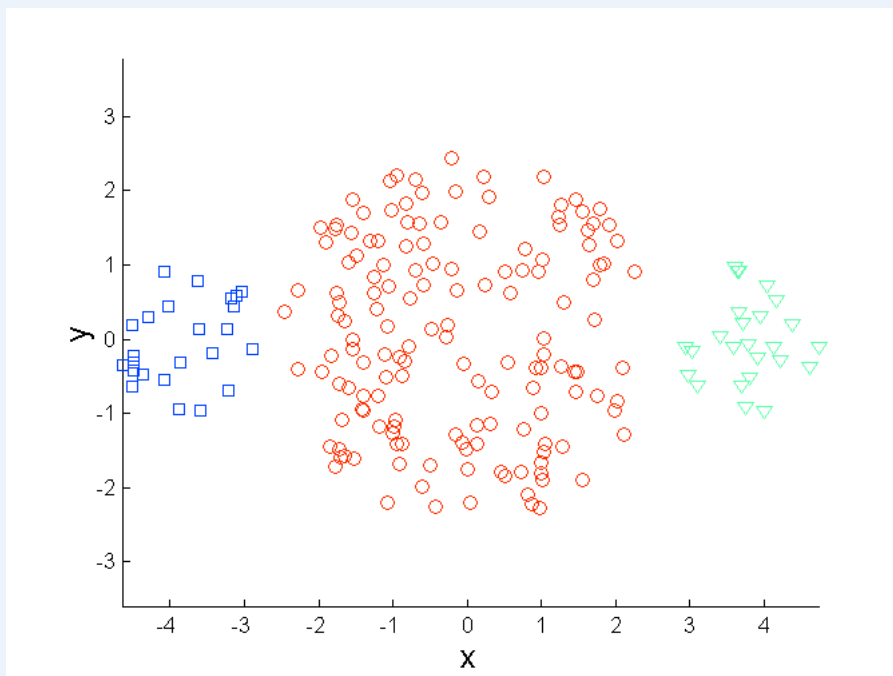
K-means的局限: 非球形



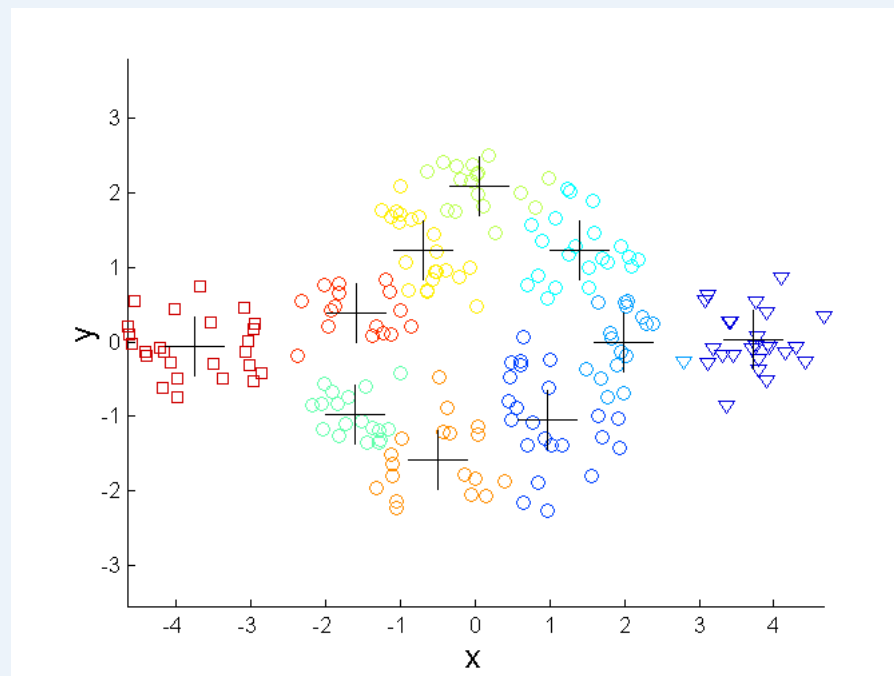
Original Points

K-means (2 Clusters)

克服K-means的局限性



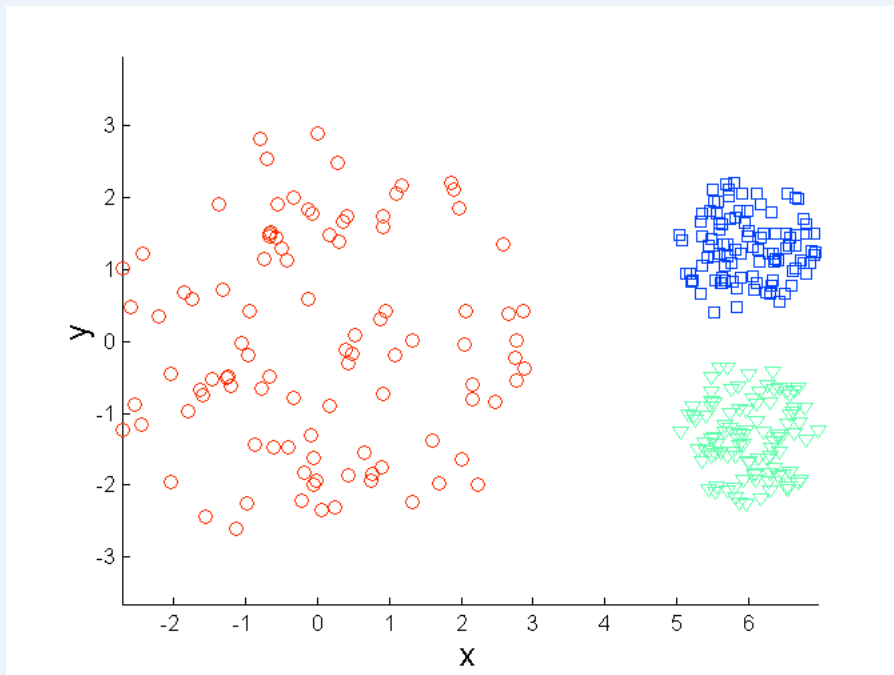
Original Points



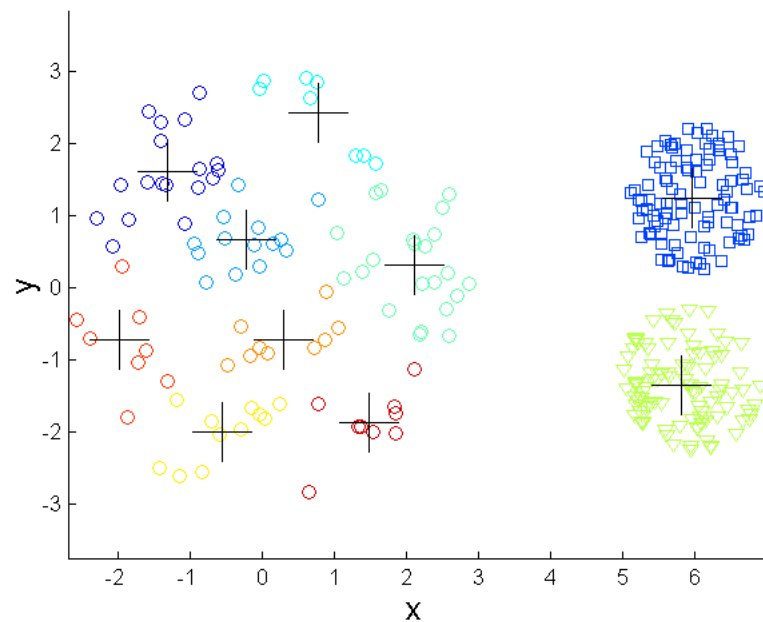
K-means Clusters

One solution is to use many clusters.
Find parts of clusters, but need to put together.

克服K-means的局限性

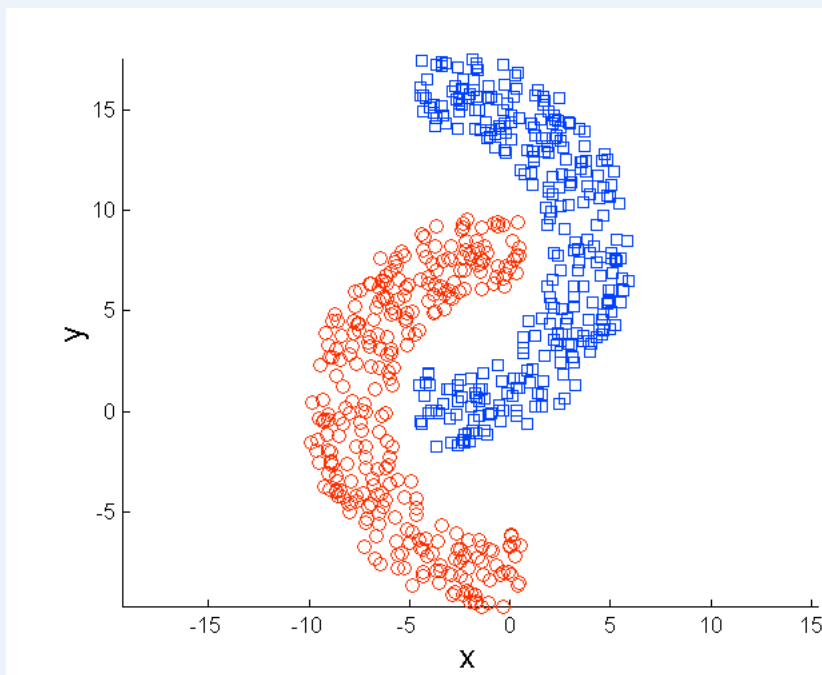


Original Points

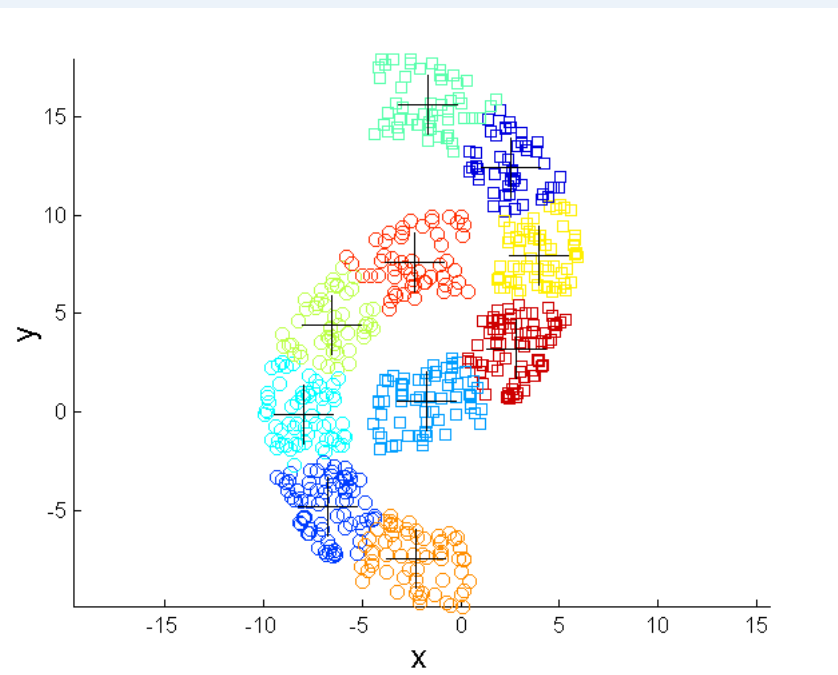


K-means Clusters

克服K-means的局限性



Original Points



K-means Clusters

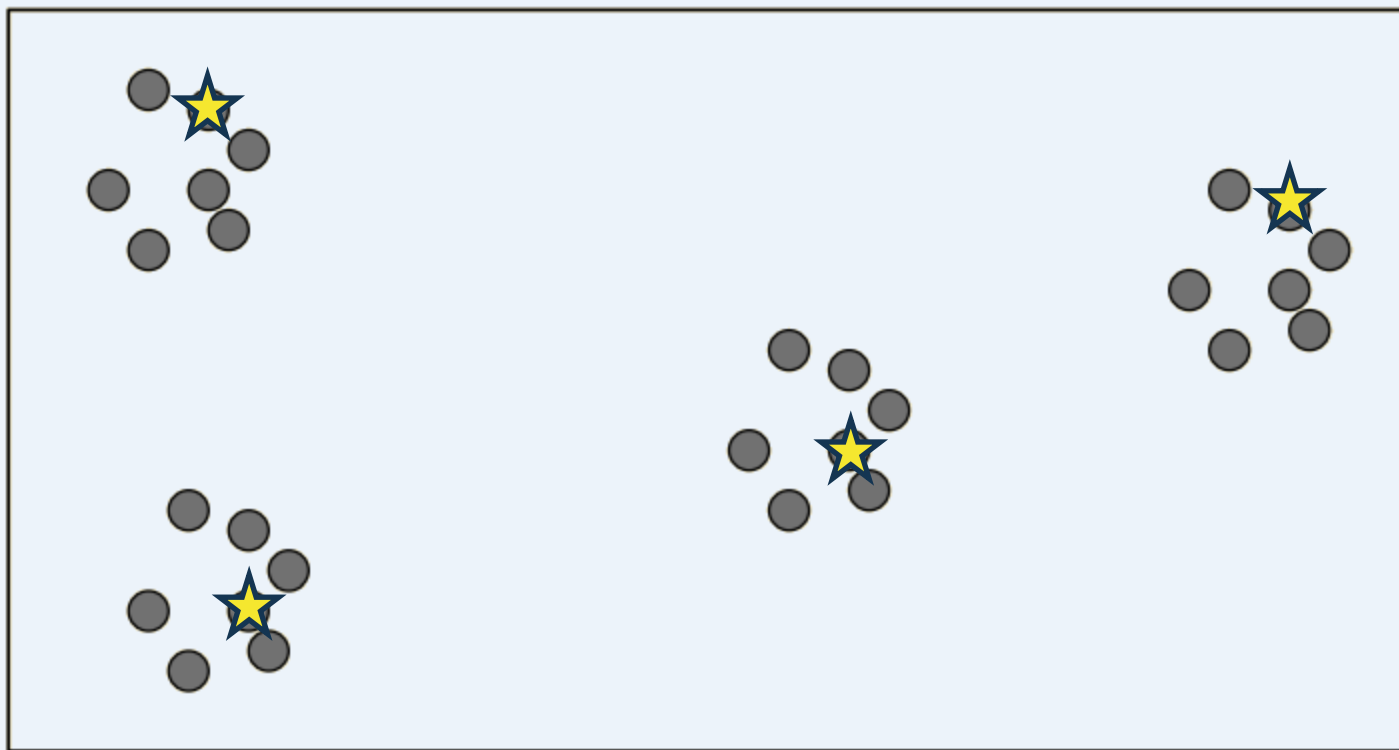
K-means++ [Arthur et al. '07]

- 算法执行步骤：（核心思想：Spreads out the centers 分散选择中心点）
 - ① 从数据集中随机均匀选取第一个中心点 x_1
 - ② 当 $2 \leq i \leq k$ 时重复以下过程：
 - 按照特定概率分布采样得到新中心点 x'
 - 该分布与 $D(x)^2$ 成正比，其中 $D(x)$ 表示数据点 x 到已选中心点的最小距离

$$\frac{D(x')^2}{\sum_{x \in \mathcal{X}} D(x)^2}$$

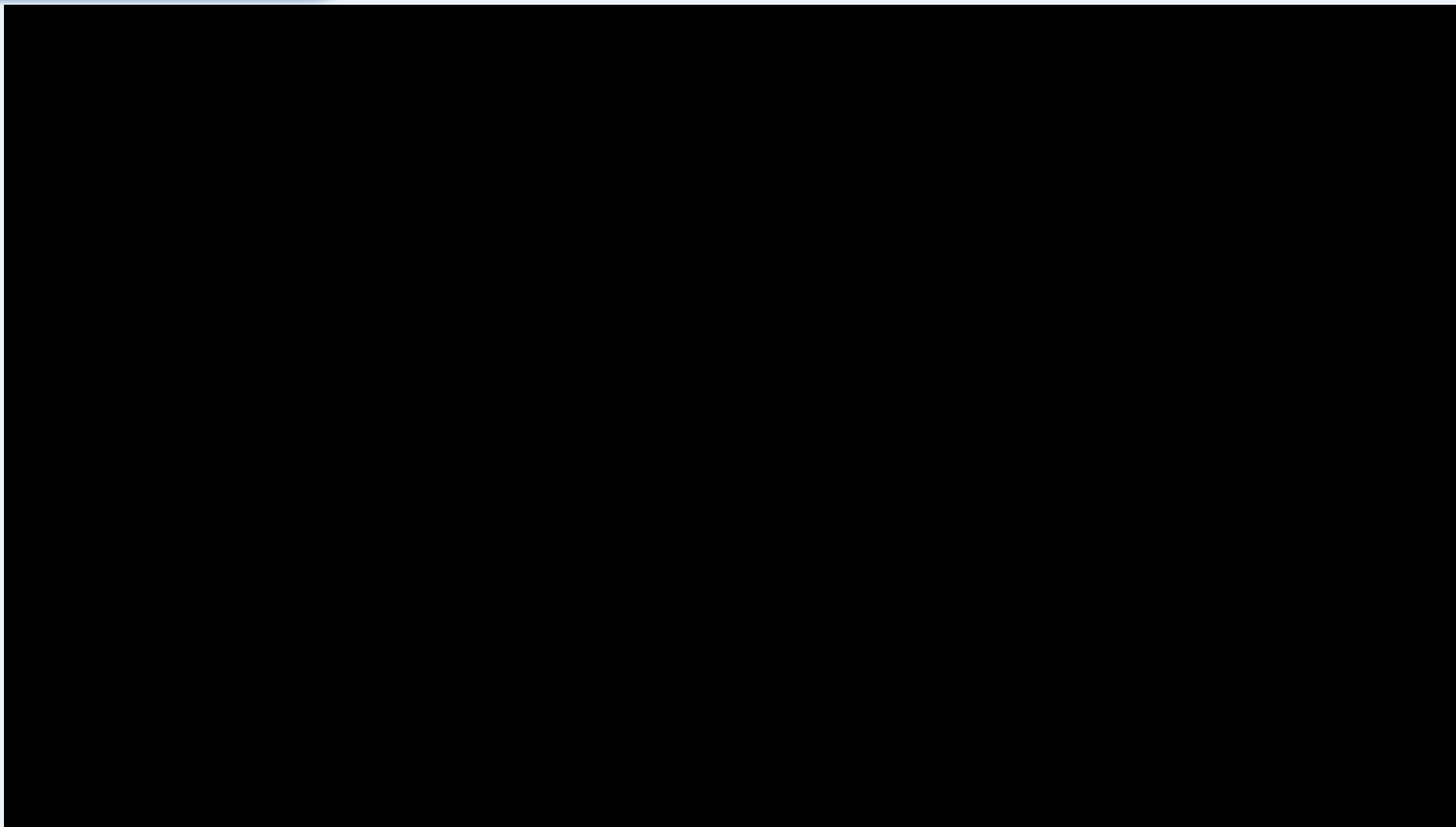
用通俗的话来讲，这个算法其实就是想让初始的几个中心点尽可能“离得越远越好”，不要扎堆。

K-means++ [Arthur et al. '07]



$$\frac{D(x')^2}{\sum_{x \in \mathcal{X}} D(x)^2}$$

K-means++



K-means++ [Arthur et al. '07]

- What's **wrong** with K-means++?
 - 需要执行K次数据遍历
 - 在大规模数据场景中，不仅数据体量庞大，K值通常也很大（例如轻易达到1000量级）
 - 可扩展性差！

可伸缩K-means: K-means||

● Intuition of

- K-means++ 每轮迭代只采样一个中心点并立即更新概率分布
- 如果我们通过独立采样扩大单次采样量会怎样？
- 这相当于大幅降低分布更新的频率
- 采用更粗粒度的采样方式
- 实践证明这种方式完全可行：这就是 K-means|| 算法的核心思想

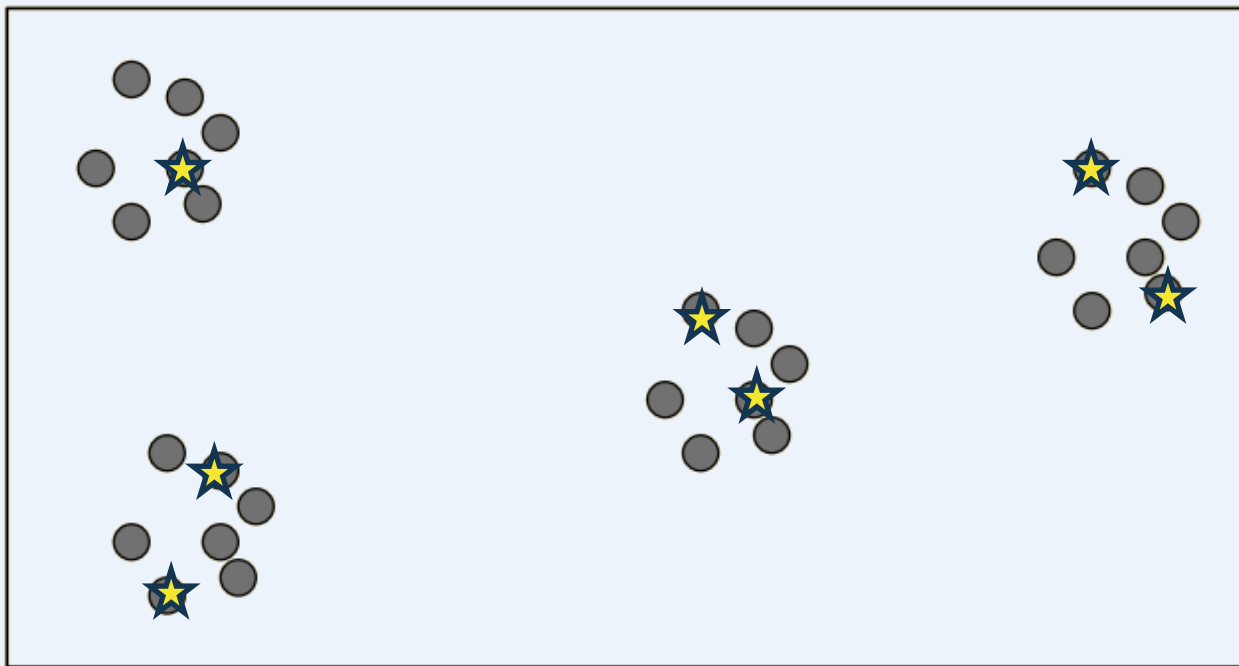
可伸缩K-means: K-means||

- 主要思路在于改变每次遍历时的取样策略，并非按照 k-means++ 那样每次遍历只取样一个样本，而是每次遍历取样A个样本，重复该取样过程大约 B次，共得到 $A*B$ 个样本点组成的集合
- 然后再聚类这 $A*B$ 个点成 k 个点，最后将这 k 个点作为初始聚类中心
- 实际实验证明**一般5次重复取样就可以得到一个较好的聚类初始中心**

可伸缩K-means: K-means||

- K-means|| Initialization

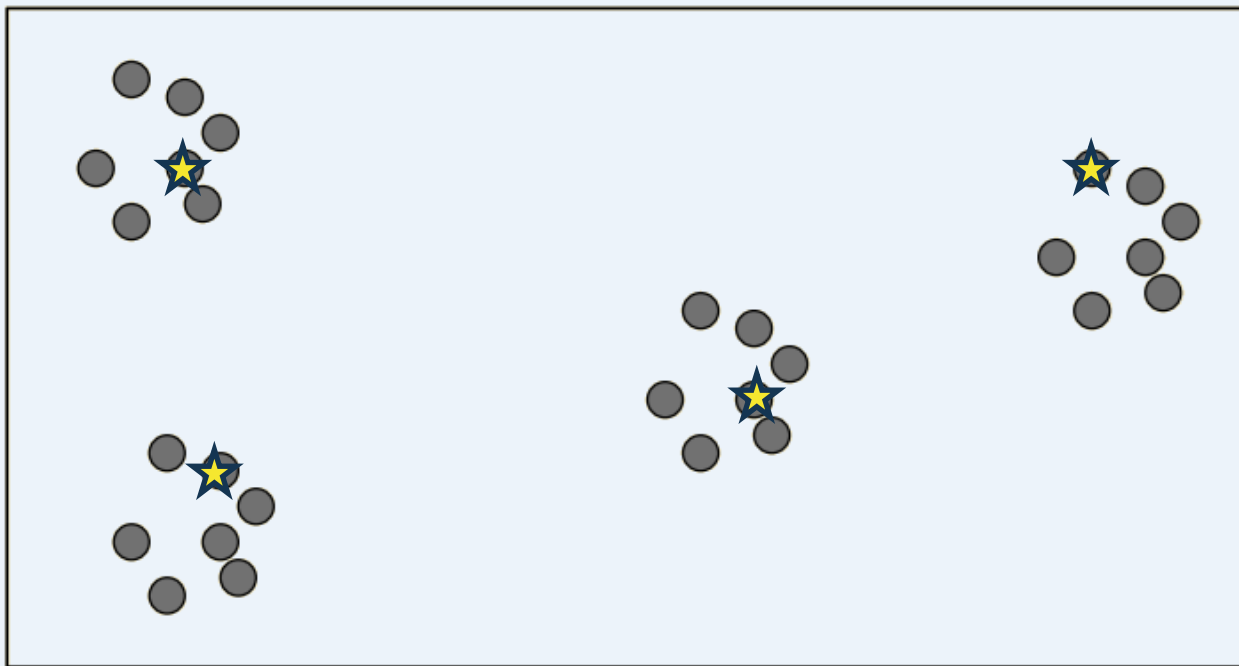
$K=4$,
Oversampling factor = 3



可伸缩K-means: K-means||

- K-means|| Initialization

$K=4$,
Oversampling factor = 3



Cluster the intermediate centers

几类kmeans算法的理论保障

- K-means, K-means++, and K-means||

Number of
iterations (R)



$R=k$: Simulating K-means++ ($l=1$) $\rightarrow$ Strong guarantee



Small R : K-means || $\rightarrow$ Can it possibly give any guarantees?



$R=0$: Lloyd $\rightarrow$ No guarantees

可伸缩K-means

● Theorem

- Optimal clustering: the sum of the squared distances between each point and its closest center is minimal
 - Solving this problem exactly is NP-hard
- Let ψ = cost of initial clustering, ψ' = cost of clustering after convergence, OPT = cost of the optimal clustering
- Using K-means++ for clustering the intermediate centers, the overall approximation factor = $O(\log k)$
 - It means $\psi' = O(\log k) * OPT$
 - **也就是说最终确定的质心能够使cost达到最优聚类的cost的 $O(\log k)$ 倍**
- K-means|| produces a constant-factor approximation to OPT , using only $O(\log(\psi/OPT))$ iterations

可伸缩K-means

● Experimental Results: Quality

设 $C = \{c_1, \dots, c_k\}$ 为一组点集, 且令 $Y \subseteq X$ 。

我们定义 Y 相对于 C 的成本为:

$$\phi_Y(C) = \sum_{y \in Y} d^2(y, C) = \sum_{y \in Y} \min_{i=1, \dots, k} \|y - c_i\|^2$$

	Clustering Cost Right After Initialization	Clustering Cost After Convergence
Random	NA	22,000
K-means++	430	65
K-means	16	14

GAUSSMIXTURE: 10,000 points in 15 dimensions

K=50

Costs scaled down by 10^4

可伸缩K-means

- Experimental Results: Convergence 收敛性
 - K-means|| reduces number of iterations even more than K-means++

	Iterations After Initialization
Random	167
K-means++	42
K-means	28

SPAM: 4,601 points in 58 dimensions
K=50

K-means 练习

- 假设有如下八个点：(1, 2), (2, 4), (1, 9), (6, 5), (4, 2), (7, 2), (8, 2), (4, 3), 使用k-Means算法对其进行聚类。设K=2, 初始聚类两个中心点 坐标分别为(1, 2), (8, 2), 算法使用曼哈顿距离 (L1 距离) 作为距离度量。请计算一次迭代后的聚类中心坐标, 并写出上述八个点分别属于哪个簇 (cluster)。

对于点(1, 2):

$$d_{11} = |1 - 1| + |2 - 2| = 0, \quad d_{12} = |1 - 8| + |2 - 2| = 7。$$

因此, (1, 2)应当被聚为第 1 类;

对于点(2, 4):

$$d_{21} = |2 - 1| + |4 - 2| = 3, \quad d_{22} = |2 - 8| + |4 - 2| = 8。$$

因此, (2, 4)应当被聚为第 1 类;

对于点(1, 9):

$$d_{31} = |1 - 1| + |9 - 2| = 7, \quad d_{32} = |1 - 8| + |9 - 2| = 14。$$

因此, (1, 9)应当被聚类为第 1 类;

对于点(6, 5):

$$d_{41} = |6 - 1| + |5 - 2| = 8, \quad d_{42} = |6 - 8| + |5 - 2| = 5。$$

因此, (6, 5)应当被聚类为第 2 类;

K-means 练习

- 假设有如下八个点：(1, 2), (2, 4), (1, 9), (6, 5), (4, 2), (7, 2), (8, 2), (4, 3), 使用k-Means算法对其进行聚类。设K=2, 初始聚类两个中心点 坐标分别为(1, 2), (8, 2), 算法使用曼哈顿距离 (L1 距离) 作为距离度量。请计算一次迭代后的聚类中心坐标, 并写出上述八个点分别属于哪个簇 (cluster) 。

对于点(4, 2):

$$d_{51} = |4 - 1| + |2 - 2| = 3, \quad d_{52} = |4 - 8| + |2 - 2| = 4。$$

因此, 点(4, 2)应当被聚类为第 1 类;

对于点(7, 2):

$$d_{61} = |7 - 1| + |2 - 2| = 6, \quad d_{62} = |7 - 8| + |2 - 2| = 1。$$

因此, 点(7, 2)应当被聚类为第 2 类;

对于点(8, 2):

$$d_{71} = |8 - 1| + |2 - 2| = 7, \quad d_{72} = |8 - 8| + |2 - 2| = 0。$$

因此, 点(8, 2)应当被聚类为第 2 类;

对于点(4, 3):

$$d_{81} = |4 - 1| + |3 - 2| = 4, \quad d_{82} = |4 - 8| + |3 - 2| = 5。$$

因此, 点(4, 3)应当被聚类为第 1 类。

综上, 类 1 的点有(1,2) (2,4) (1,9) (4,2) (4,3), 则对应的聚类中心为: (2.4,4)。

类 2 的点有(6,5) (7,2) (8,2) , 则对应的聚类中心为: (7,3)。

DBSCAN

- DBSCAN是一种基于密度（Density-based）的聚类算法.

密度Density: 指定半径范围内包含的数据点数量

核心点core point: 在半径范围内包含至少MinPts个点的点

- 这类点位于簇的内部核心区域

边界点border point: 在半径范围内包含少于MinPts个点，但位于某个核心点的邻域内

- 处于簇的边界区域

噪声点noise point: 既不是核心点也不是边界点的点

- 属于异常数据或离散点

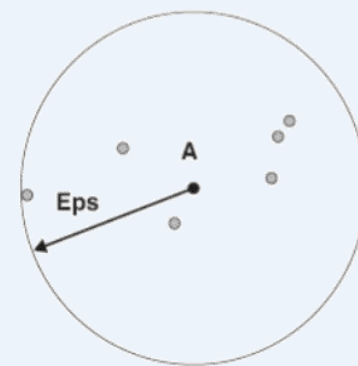


Figure 8.20. Center-based density.

- **Density** = number of points within a specified radius (Eps)

DBSCAN: 核心点、边界点、噪声点

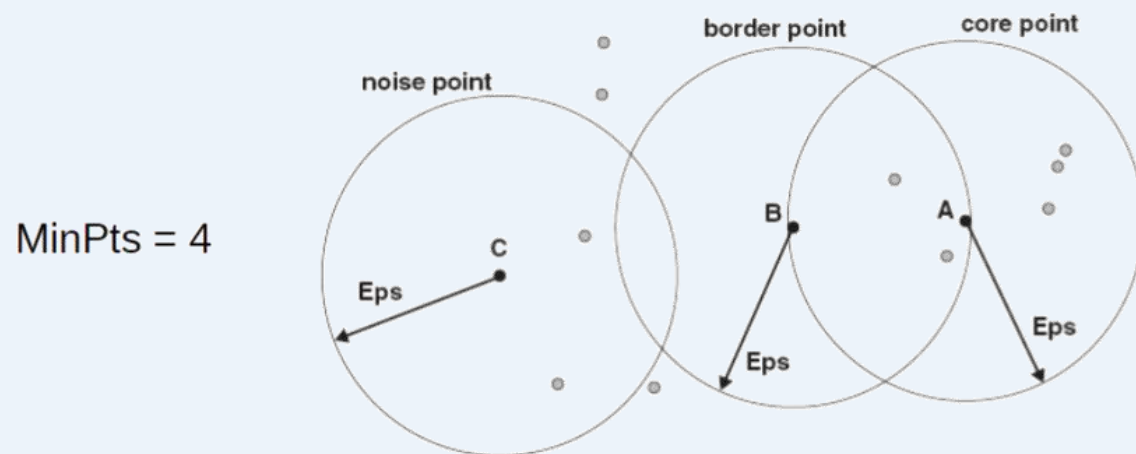
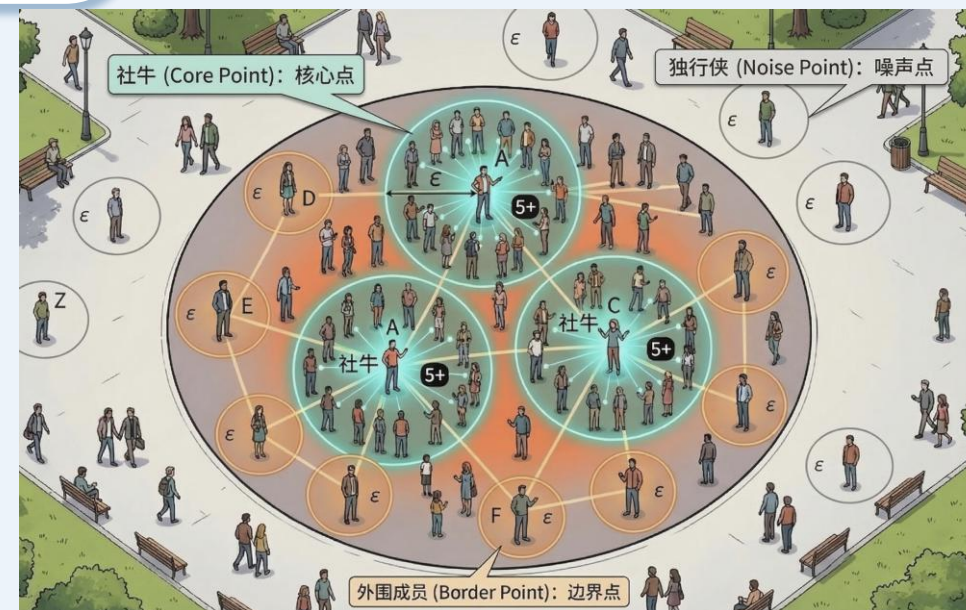


Figure 8.21. Core, border, and noise points.

- A point is a **core point** if it has more than a specified number of points (MinPts) within Eps. These are points that are at the interior of a cluster
- A **border point** has fewer than MinPts within Eps, but is in the neighborhood of a core point
- A **noise point** is any point that is not a core point or a border point.

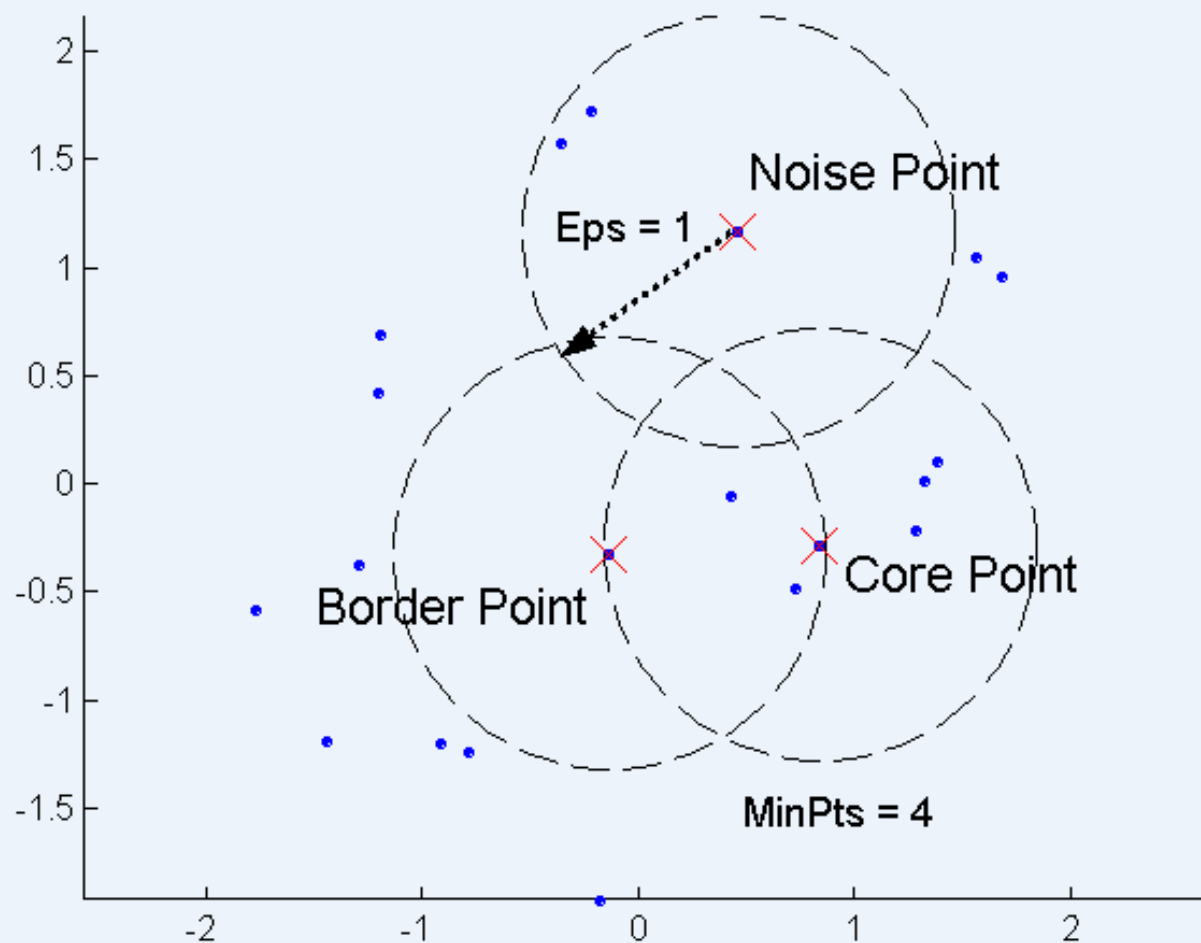


核心点

边界点

噪声点

DBSCAN: 核心点、边界点、噪声点



DBSCAN 算法

● Center-based approach

算法 S.4 DBSCAN 算法步骤

1. 将所有点标记为**核心点**、**边界点**或**噪声点**。
2. **剔除噪声点**（使其不参与后续聚类）。
3. 在所有**相互距离在 Eps 范围内的核心点**之间建立连接边。
4. 将每个**连通的**核心点组形成一个独立的簇。
5. 将每个**边界点**分配到与其关联的**核心点所在的簇**中。

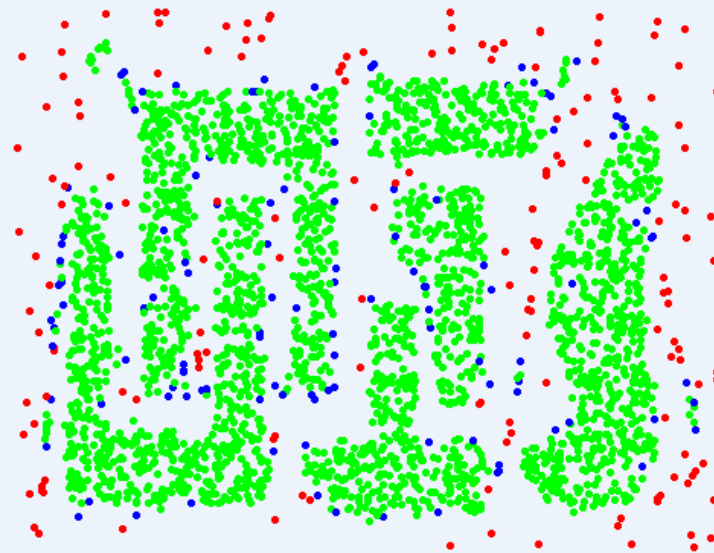
Algorithm 8.4 DBSCAN algorithm.

- 1: Label all points as core, border, or noise points.
 - 2: Eliminate noise points.
 - 3: Put an edge between all core points that are within Eps of each other.
 - 4: Make each group of connected core points into a separate cluster.
 - 5: Assign each border point to one of the clusters of its associated core points.
-

DBSCAN: 核心点、边界点、噪声点



Original Points



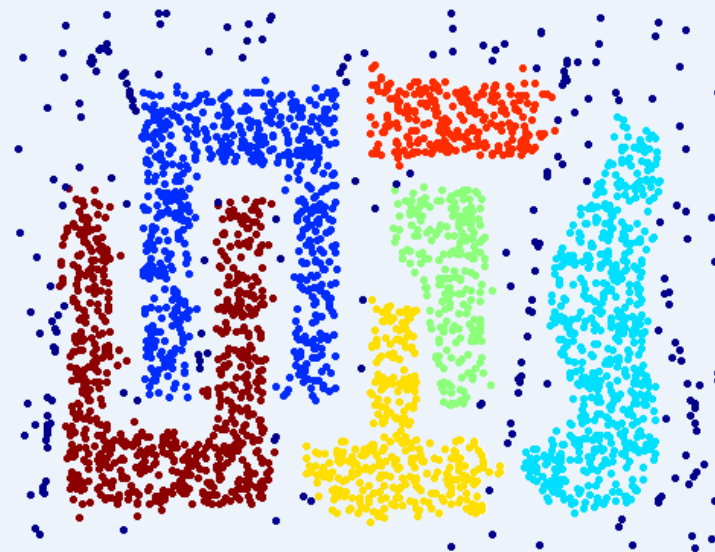
Point types: core,
border and noise

Eps = 10, MinPts = 4

DBSCAN擅长的情况



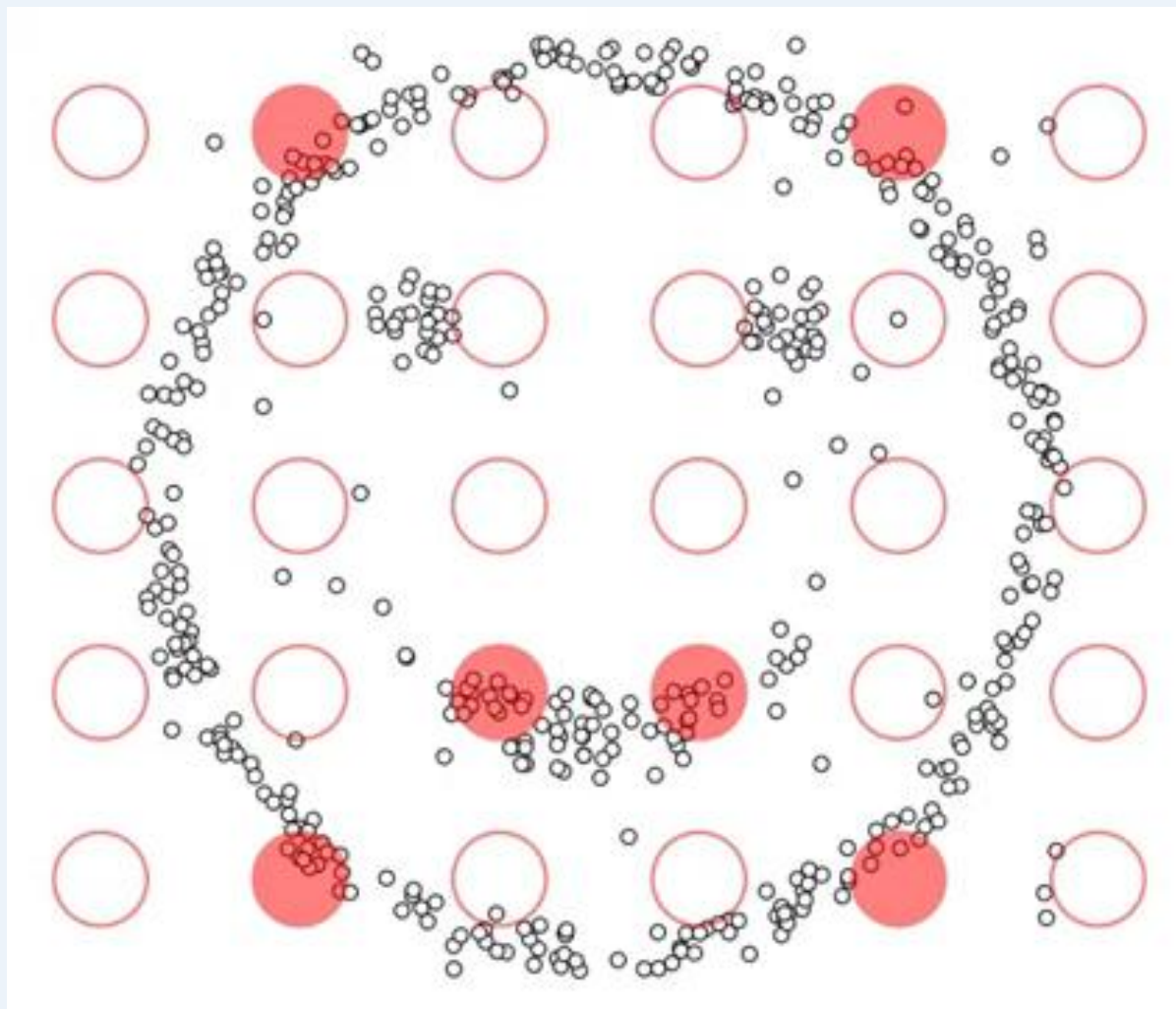
Original Points



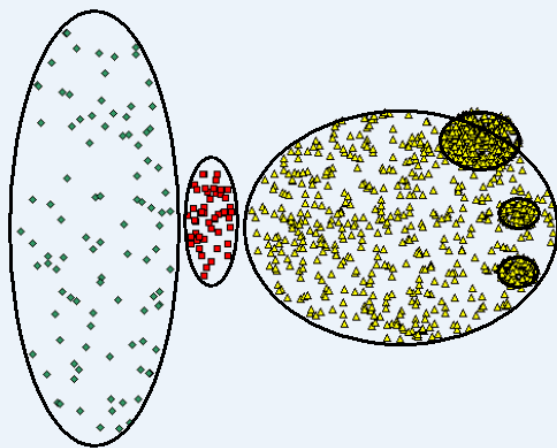
Clusters

- Resistant to Noise
- Can handle clusters of different shapes and sizes

DBSCAN



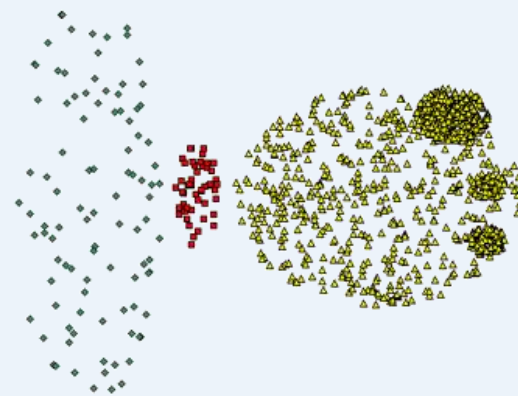
DBSCAN不擅长的情况



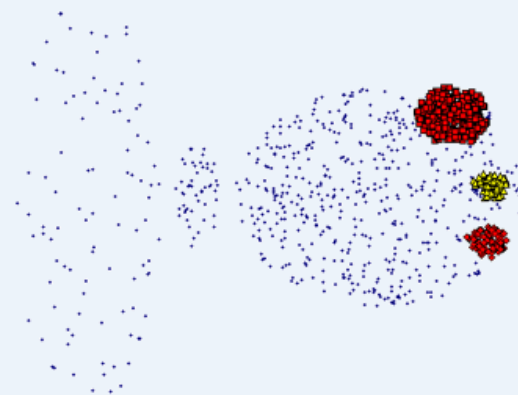
Original Points

- Varying densities
- High-dimensional data

维度灾难：距离失效



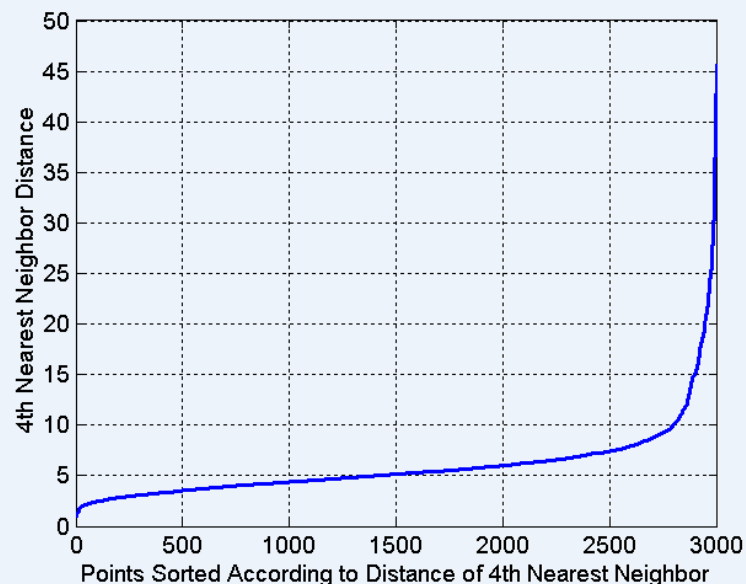
(MinPts=4, Eps=9.75).



(MinPts=4, Eps=9.92)

DBSCAN: 确定EPS和MinPts

- Idea is that for points in a cluster, their k^{th} nearest neighbors are at roughly the same distance
- Noise points have the k^{th} nearest neighbor at farther distance
- So, plot sorted distance of every point to its k^{th} nearest neighbor



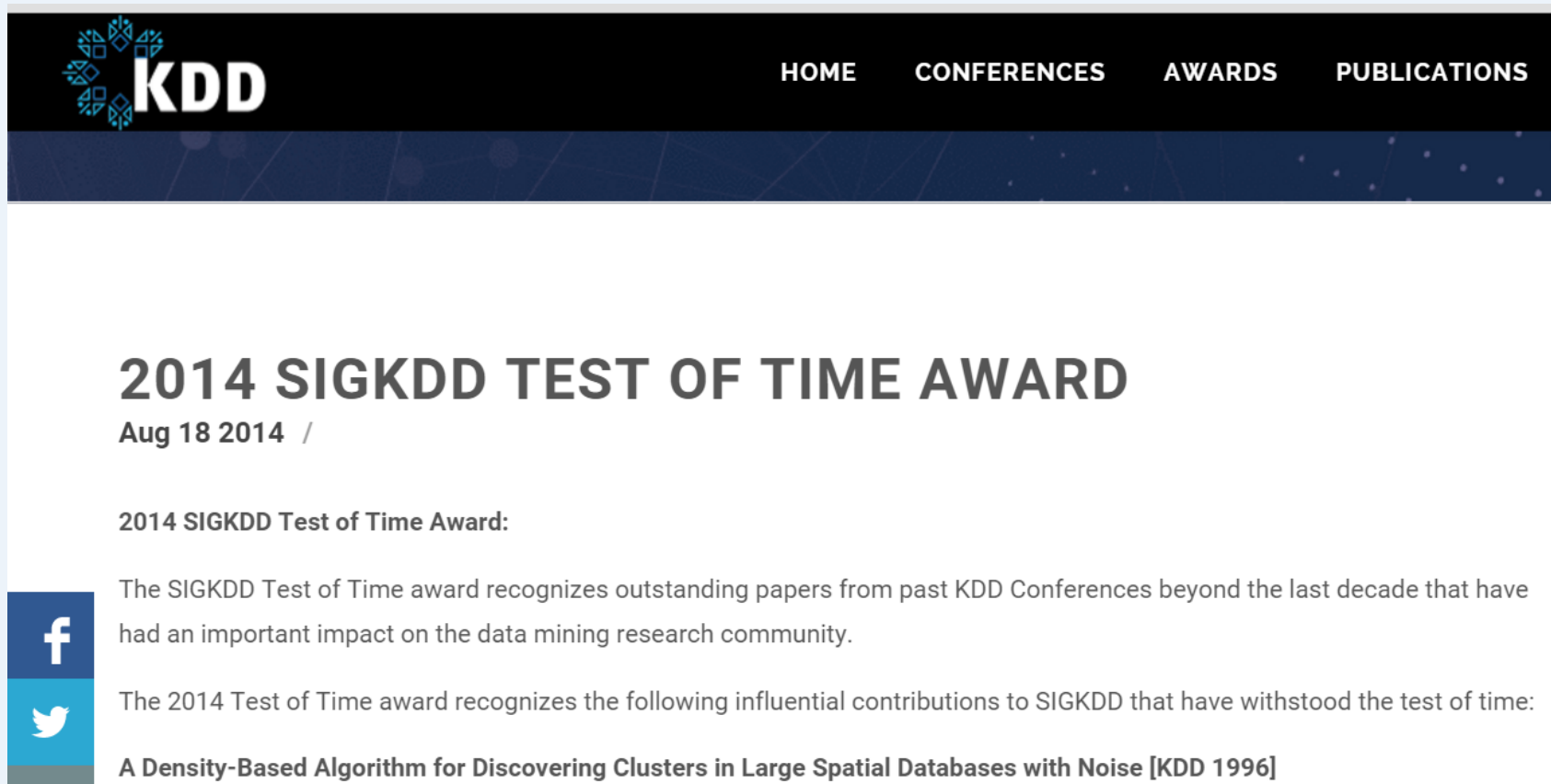
时间、空间复杂度

Time and Space Complexity

The basic time complexity of the DBSCAN algorithm is $O(m \times \text{time to find points in the } Eps\text{-neighborhood})$, where m is the number of points. In the worst case, this complexity is $O(m^2)$. However, in low-dimensional spaces, there are data structures, such as kd-trees, that allow efficient retrieval of all points within a given distance of a specified point, and the time complexity can be as low as $O(m \log m)$. The space requirement of DBSCAN, even for high-dimensional data, is $O(m)$ because it is only necessary to keep a small amount of data for each point, i.e., the cluster label and the identification of each point as a core, border, or noise point.

2014 KDD TEST OF TIME AWARD

- A Density-Based Algorithm for Discovering Clusters in Large Spatial Databases with Noise [KDD 1996]



The screenshot shows the official announcement for the 2014 SIGKDD Test of Time Award. The header features the KDD logo and navigation links for HOME, CONFERENCES, AWARDS, and PUBLICATIONS. The main title is "2014 SIGKDD TEST OF TIME AWARD" with the date "Aug 18 2014". Below this, it states "2014 SIGKDD Test of Time Award:" followed by a paragraph explaining the award's purpose. Social media icons for Facebook and Twitter are present. The bottom section lists the winning paper: "A Density-Based Algorithm for Discovering Clusters in Large Spatial Databases with Noise [KDD 1996]".

KDD HOME CONFERENCES AWARDS PUBLICATIONS

2014 SIGKDD TEST OF TIME AWARD

Aug 18 2014 /

2014 SIGKDD Test of Time Award:

The SIGKDD Test of Time award recognizes outstanding papers from past KDD Conferences beyond the last decade that have had an important impact on the data mining research community.

The 2014 Test of Time award recognizes the following influential contributions to SIGKDD that have withstood the test of time:

A Density-Based Algorithm for Discovering Clusters in Large Spatial Databases with Noise [KDD 1996]

2014 KDD TEST OF TIME AWARD

- **A Density-Based Algorithm for Discovering Clusters in Large Spatial Databases with Noise [KDD 1996]**
 - Proposed the now well-known clustering algorithm DBSCAN
 - 10000+ citations

[PDF] [A density-based algorithm for discovering clusters in large spatial databases with noise.](#)

[M Ester](#), [HP Kriegel](#), [J Sander](#), [X Xu](#) - Kdd, 1996 - [aaai.org](#)

Abstract Clustering algorithms are attractive for the task of class identification in spatial databases. However, the application to large spatial databases rises the following requirements for clustering algorithms: minimal requirements of domain knowledge to determine the input parameters, discovery of clusters with arbitrary shape and good efficiency on large databases. The well-known clustering algorithms offer no solution to ...

被引用次数: 10010 相关文章 所有 75 个版本 引用 保存 更多

2014 KDD TEST OF TIME AWARD

- **A Density-Based Algorithm for Discovering Clusters in Large Spatial Databases with Noise [KDD 1996]**
 - Claimed that the average run time complexity of DBSCAN is $O(m \cdot \log m)$
 - Follow papers mis-claimed that the complexity of DBSCAN is $O(m^2 \cdot \log m)$

2015 SIGMOD Best Paper Award

- DBSCAN Revisited: Mis-Claim, Un-Fixability, and Approximation.

SIGMOD Best Paper Award

Recipients of the award are the following:

2015	DBSCAN Revisited: Mis-Claim, Un-Fixability, and Approximation. Yufei Tao, Junhao Gan
2014	Materialization Optimizations for Feature Selection Workloads. Ce Zhang, Arun Kumar, Christopher Ré
2013	Massive Graph Triangulation. Xiaocheng Hu, Yufei Tao, Chin-Wan Chung
2012	High-Pe 2013 SIGMOD Best Paper Award Processing over XML Streams. Barzan Mozafari, Kai Zeng, Carlo Zaniolo
2011	Entangled Queries: Enabling Declarative Data-Driven Coordination. Nitin Gupta, Lucja Kot, Sudip Roy, Gabriel Bender, Johannes Gehrke, Christoph Koch
2010	FAST: Fast Architecture Sensitive Tree Search on Modern CPUs and GPUs. Changkyu Kim, Jatin Chhugani, Nadathur Satish, Eric Sedlar, Anthony Nguyen, Tim Kaldewey, Victor Lee, Scott Brandt, and Pradeep Dubey [award citation]

SIGMOD/PODS 2015

[Website](#)

[Facebook](#)

RSS

[RSS](#) [SIGMOD News](#)

SIGMOD Record

**SIGMOD
RECORD**

Web

2015 SIGMOD Best Paper Award

- DBSCAN Revisited: Mis-Claim, Un-Fixability, and Approximation.
 - For $d \geq 3$, DBSCAN can't have $O(m \cdot \log m)$ time complexity.
 - Prove that the DBSCAN problem requires $\Omega(m^{4/3})$ time to solve in $d \geq 3$.
 - Introduce a new concept called ρ -approximate DBSCAN as an alternative to DBSCAN on large datasets of $d \geq 3$.

2015 SIGMOD Best Paper Award

Yufei Tao



Professor

Room 633, Building 78
[School of Information Technology and Electrical Engineering](#)
[University of Queensland](#)
Brisbane, Queensland 4072
Australia

Tel: +61-7-33658319

Email: taoyf@itee.uq.edu.au

[Publications](#) and [Google Scholar](#)

General

Short Bio

Yufei Tao joined as a Professor the School of Information Technology and Electrical Engineering, the University of Queensland (UQ) in Jan 2016. He obtained his PhD degree from Hong Kong University of Science and Technology (HKUST) in 2002, proudly under the supervision of Prof. Dimitris Papadias. He was a Visiting Scientist at the Carnegie Mellon University during 2002-2003, and then an Assistant Professor at the City University of Hong Kong during 2003-2006, after which he worked till Jan 2016 at the Chinese University of Hong Kong (CUHK), where he was promoted to Professor in 2009. From 2011 to 2013, he was simultaneously a Visiting Professor, under the World Class University program of the Korean government, in the Korea Advanced Institute of Science and Technology (KAIST), Korea. Currently, he also holds an honorary position of Adjunct Professor in the Department of Computer Science and Engineering, CUHK.

He served as an associate editor of *ACM Transactions on Database Systems (TODS)* from 2008 to 2015, and of *IEEE Transactions on Knowledge and Data Engineering (TKDE)* from 2012 to 2014. He served as a PC co-chair of *International Conference on Data Engineering (ICDE)* 2014, and of *International Symposium on Spatial and Temporal Databases (SSTD)* 2011. He gave a keynote speech at *International Conference on Database Theory (ICDT)* 2016.

Major Awards

SIGMOD Best Paper Award 2015
SIGMOD Best Paper Award 2013
Hong Kong Young Scientist Award 2002

2015 SIGMOD Best Paper Award

- 怀疑的精神
- 善于思考，善于发现问题
- 科学研究的发展是在曲折中不断前进的！



曲折中前进
螺旋式上升

Reference

- Ester, Martin, et al. "A density-based algorithm for discovering clusters in large spatial databases with noise." *Kdd*. Vol. 96. No. 34. 1996.
- Gan, Junhao, and Yufei Tao. "DBSCAN Revisited: Mis-Claim, Un-Fixability, and Approximation." *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data*. ACM, 2015.
- 如何评价 SIGMOD 2015 最佳论文《DBSCAN Revisited》？
<http://www.zhihu.com/question/31845624>

K-means和DBSCAN的比较

- 两者都是将每个对象分配到单一簇的分区聚类
 - K-means: all object 所有对象
 - DBSCAN: discard noise 剔除噪声
- Type
 - K-means: prototype原型（簇类中心）-based notion
 - DBSCAN: density-based notion
- Both perform poorly with widely differing densities两者在密度差异较大时表现不佳
 - K-means: has difficult with non-globular cluster and clusters with different size难以处理非球形簇和不同大小的簇
 - DBSCAN: can handle clusters of different size and shapes, not affected by noise 能够处理不同大小和形状的簇，不受噪声影响

K-means和DBSCAN的比较

- Definition requirement
 - K-means: need well-defined centroid 需要定义好的中心结构
 - DBSCAN: need definition of density 需要定义好的密度
- Sparse, high-dimensional data
 - K-means: 在稀疏高维数据上表现良好，例如文档数据聚类，适合处理像文本这样的高维数据集。
 - DBSCAN: 在稀疏高维数据上使用欧几里得距离定义密度时表现较差，因高维空间的“维度诅咒”导致密度计算不准确
- Distribution of data
 - K-means: spherical Gaussian distributions 球形高斯分布
 - DBSCAN: no assumption 无假设，灵活度更好

K-means和DBSCAN的比较

- Both using all attributes
- Number of clusters
 - K-means: user define
 - DBSCAN: automatically,
Two parameters: Eps and Minpts

方面	K-means	DBSCAN
定义需求	需要明确的中心点 (centroid)	需要定义密度 (eps 和 minPts)
稀疏高维数据	表现良好, 例如文档数据	使用欧几里得距离定义密度时表现较差
数据分布假设	假设球形高斯分布	无假设, 能处理不同形状和大小的簇
噪声处理	对噪声敏感, 可能影响中心点计算	能有效处理噪声, 边界点和噪声点可明确区分



簇有效性

- For cluster analysis, the analogous question is how to evaluate the “goodness” of the resulting clusters? 如何评估找出来的簇结构好坏
- But “clusters are in the eye of the beholder”! 主观性太强!
- Then why do we want to evaluate them?
 - To compare clustering algorithms
 - To compare two sets of clusters

簇评估(Cluster Validation)的方面

1. Determining the **clustering tendency** **聚类趋势** of a set of data, i.e., distinguishing whether non-random structure actually exists in the data **先看看是不是真的有簇结构存在**
2. Comparing the results of a cluster analysis to externally known information, e.g., to externally given class labels. 将聚类分析与外部已知信息（例如外部给定的类别标签）进行比较
3. Evaluating how well the results of a cluster analysis fit the data *without* reference to external information. 评估聚类分析结果对数据的拟合程度，不参考外部信息
- **Use only the data**
4. Comparing the results of two different sets of cluster analysis to determine which is better.
5. Determining the 'correct' number of clusters.

簇有效性指标

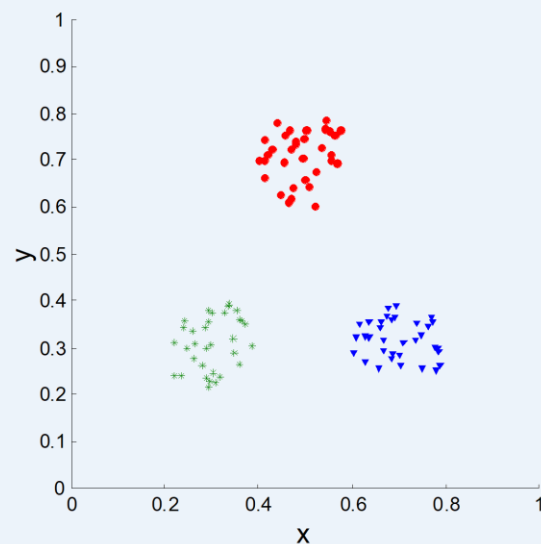
- Numerical measures that are applied to judge various aspects of cluster validity, are classified into the following three types.
 - **External Index:** Used to measure the extent to which cluster labels match externally supplied class labels. 外部指标用于比较聚类结果与外部提供的真实标签（ground truth），衡量两者的一致性
 - **Entropy 熵**
 - **Internal Index:** Used to measure the goodness of a clustering structure *without* respect to external information. 内部指标不依赖外部标签，仅使用数据本身来评估聚类质量，关注簇的紧密度（cohesion）和分离度（separation）
 - **Sum of Squared Error (SSE)**
 - **Relative Index:** Used to compare two different clusterings or clusters. 实际上不是一个单独的簇评估类型，而是度量的具体使用
 - Often an external or internal index is used for this function, e.g., SSE or entropy 例如可以用SSE比较K-means在不同k 值下的结果
- Sometimes these are referred to as **criteria** instead of **index**
- However, sometimes criterion is the general strategy and index is the numerical measure that implements the criterion. 例如 “最小化簇内方差” 是一个准则，而SSE是实现这一准则的指标

簇有效性指标：相关性

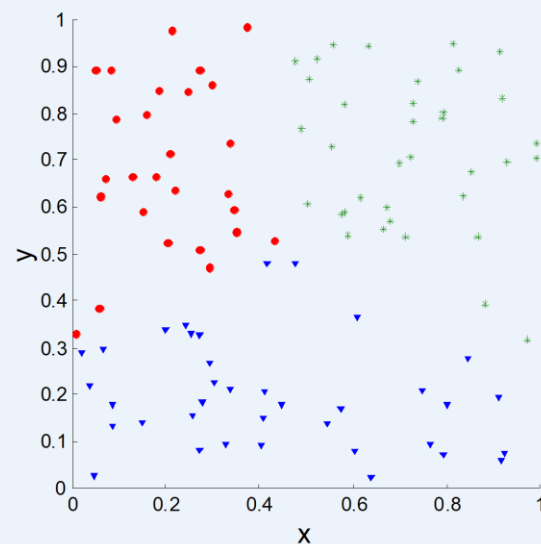
- Two matrices
 - Proximity Matrix (Similarity/Dissimilarity Matrix) 邻近矩阵
 - “Incidence” Matrix 关联矩阵
 - One row and one column for each data point
 - An entry is 1 if the associated pair of points belong to the same cluster
 - An entry is 0 if the associated pair of points belongs to different clusters
- Compute the correlation between the two matrices
 - Since the matrices are symmetric, only the correlation between $n(n-1) / 2$ entries needs to be calculated.
- High correlation indicates that points that belong to the same cluster are close to each other.
- Not a good measure for some density or contiguity based clusters.

簇有效性指标：相关性

- Correlation of incidence and proximity matrices for the K-means clusterings of the following two data sets.



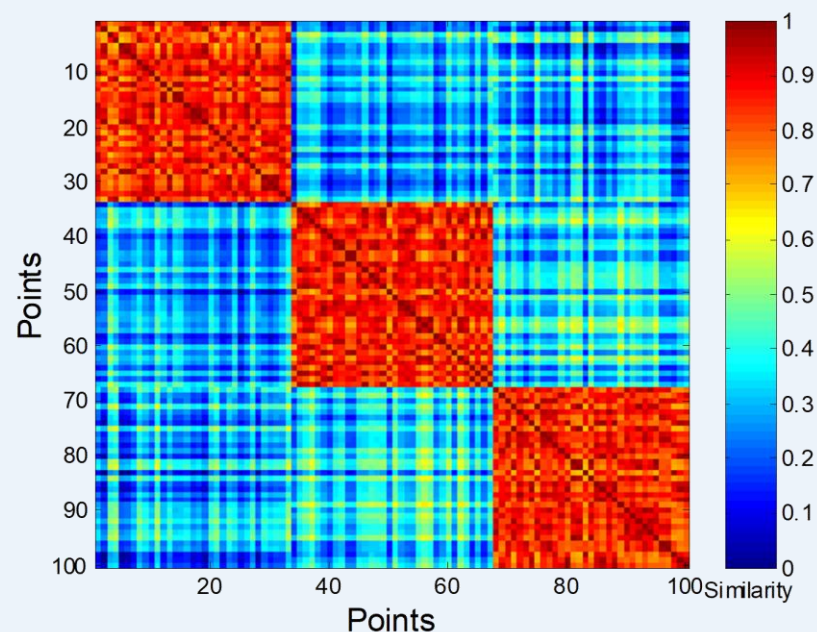
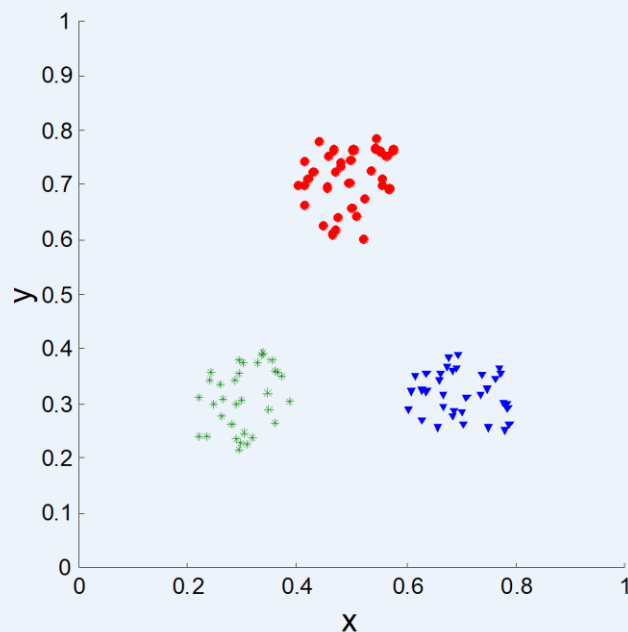
Corr = 0.9235



Corr = 0.5810

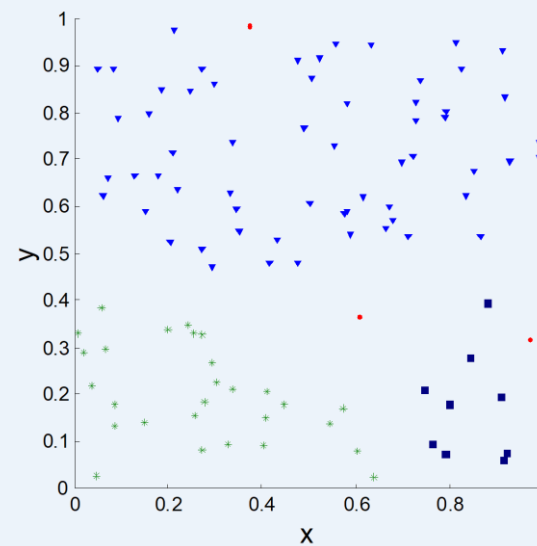
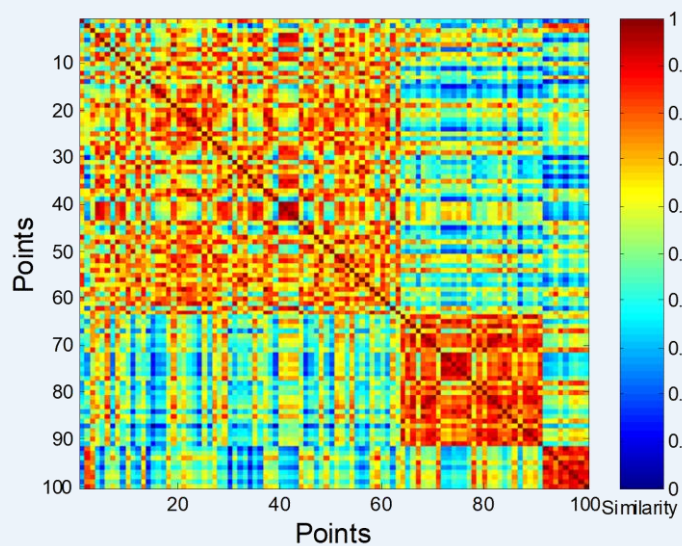
簇评估：使用相似矩阵

- Order the similarity matrix with respect to cluster labels and inspect visually.



簇评估：使用相似矩阵

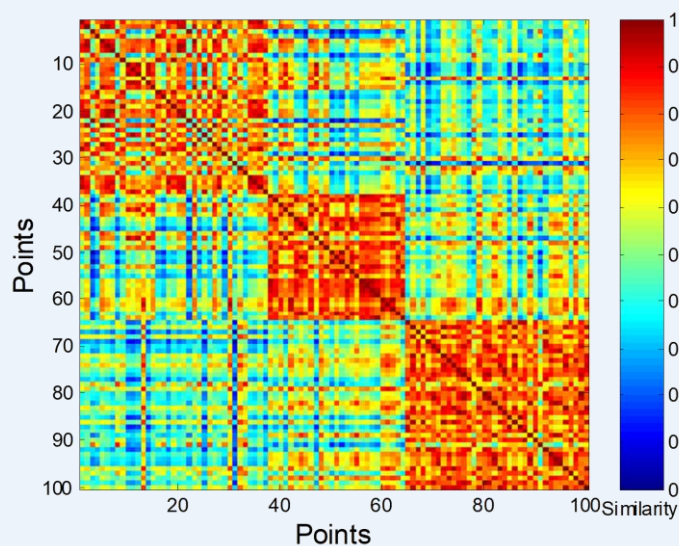
- Clusters in random data are not so crisp



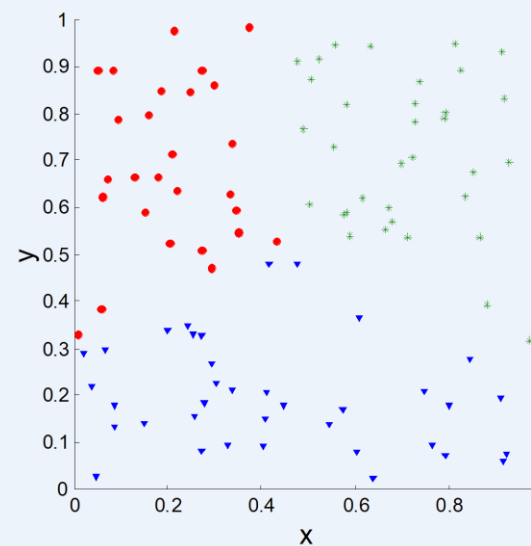
DBSCAN

簇评估：使用相似矩阵

- Clusters in random data are not so crisp

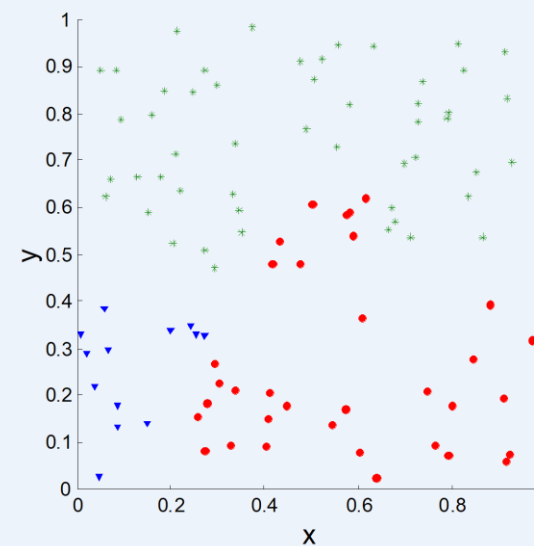
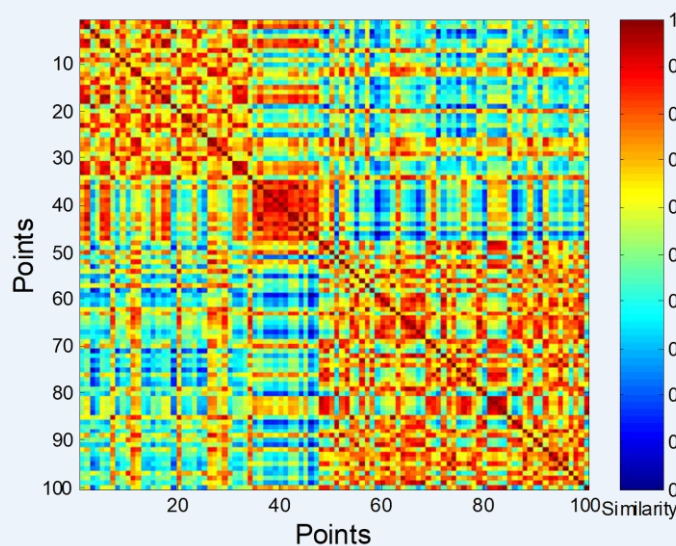


K-means



簇评估：使用相似矩阵

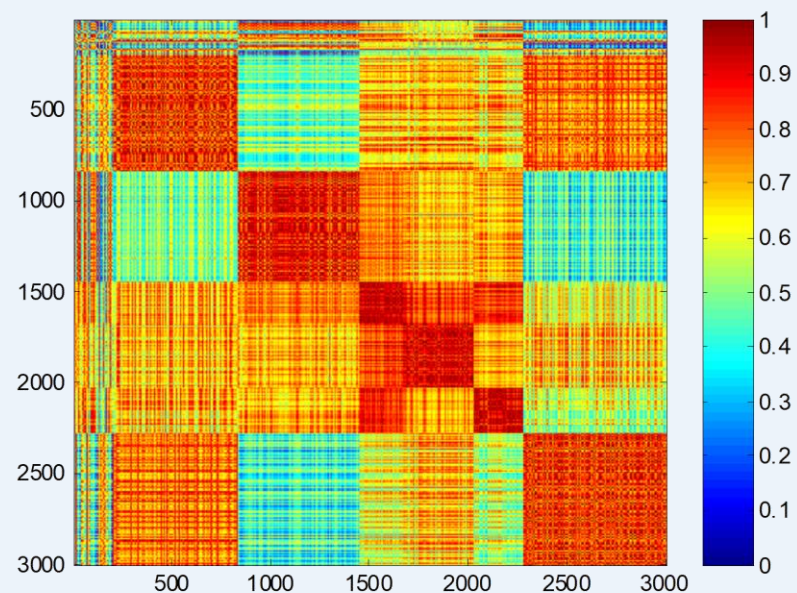
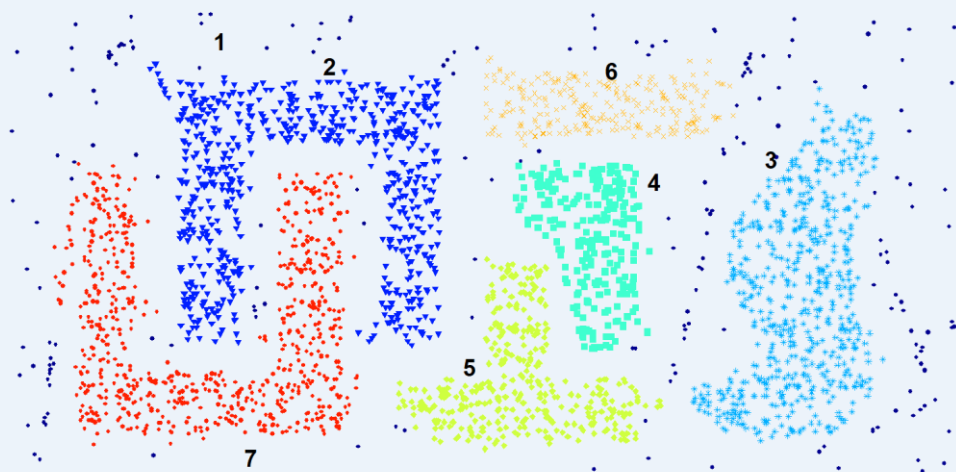
- Clusters in random data are not so crisp



Complete linkage hierarchical clustering



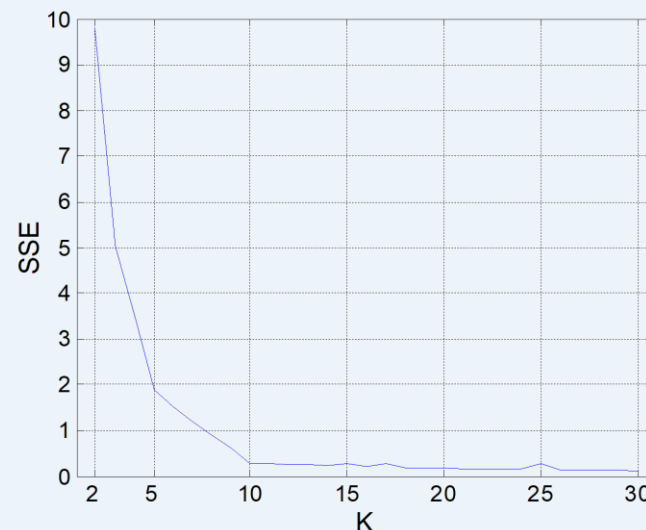
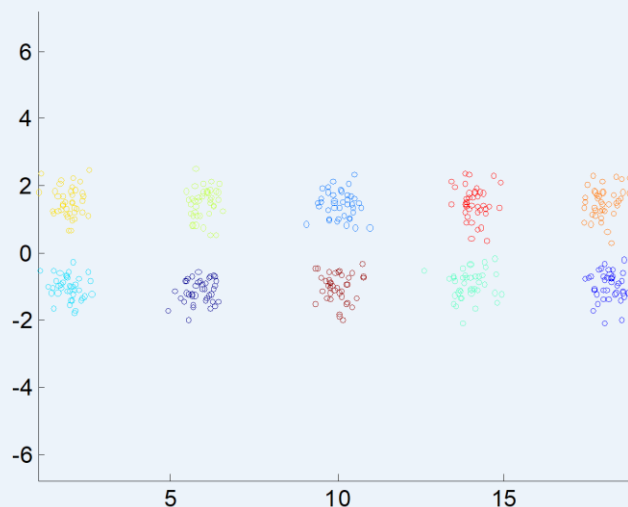
簇评估：使用相似矩阵



DBSCAN

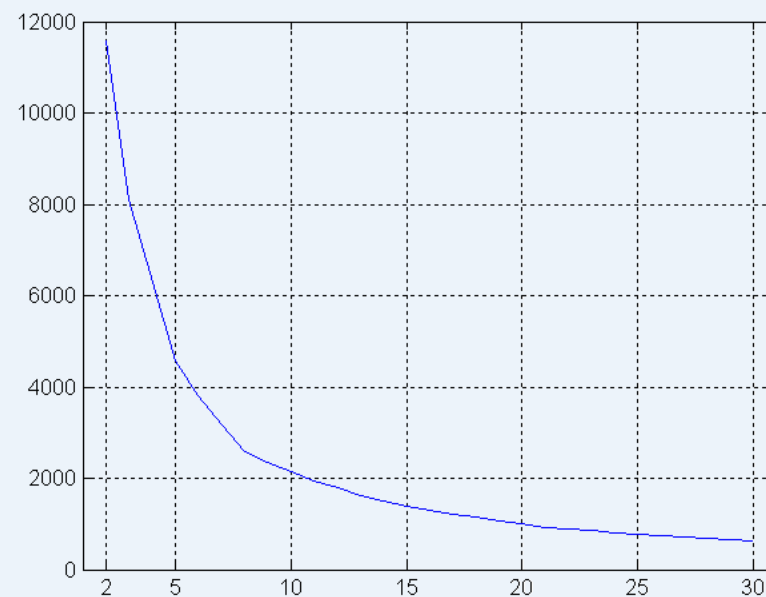
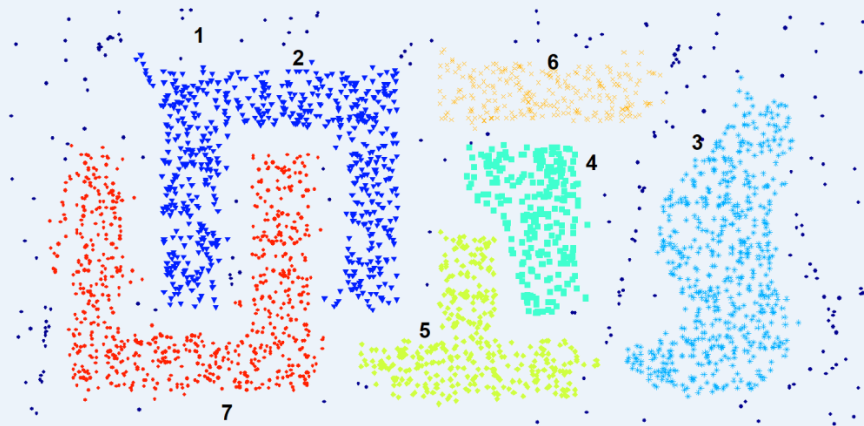
内部指标: SSE

- Clusters in more complicated figures aren't well separated
- **Internal Index**: Used to measure the goodness of a clustering structure without respect to external information
 - SSE
- SSE is good for comparing two clusterings or two clusters (average SSE).
- Can also be used to estimate the number of clusters



内部指标: SSE

- SSE curve for a more complicated data set



SSE of clusters found using K-means

内部指标: 凝聚度和分离度

- **Cluster Cohesion 凝聚度**: Measures how closely related are objects in a cluster
 - Example: SSE sum of squares within group
- **Cluster Separation 分离度**: Measure how distinct or well-separated a cluster is from other clusters
- Example: Squared Error
 - Cohesion is measured by the within cluster sum of squares (SSE)

$$SSE = \sum_i \sum_{x \in C_i} (x - m_i)^2$$

- Separation is measured by the between cluster sum of squares (SSB: sum of squares between groups)

$$SSB = \sum_i |C_i| (m - m_i)^2$$

- Where $|C_i|$ is the size of cluster i

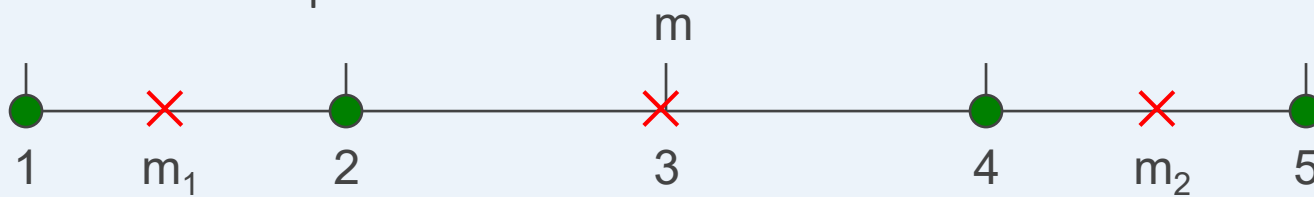
内部指标: 凝聚度和分离度

- Example:

SSE+SSB=TSS **constant**

$$TSS = \sum_x (x - m)^2$$

total sum of squares



K=1 cluster:

$$SSE = (1-3)^2 + (2-3)^2 + (4-3)^2 + (5-3)^2 = 10$$

$$SSB = 4 \times (3-3)^2 = 0$$

$$Total = 10 + 0 = 10$$

K=2 clusters:

$$SSE = (1-1.5)^2 + (2-1.5)^2 + (4-4.5)^2 + (5-4.5)^2 = 1$$

$$SSB = 2 \times (3-1.5)^2 + 2 \times (4.5-3)^2 = 9$$

$$Total = 1 + 9 = 10$$

内部指标: 凝聚度和分离度

$$SSE + SSB = TSS \quad \text{Try}$$

给定:

- x_{ij} : 第 j 个簇中的第 i 个点
- m_j : 第 j 个簇的中心
- m : 所有点的总体中心
- n_j : 第 j 个簇的点数, t 个簇, $n = \sum_{j=1}^t n_j$ 为总点数

总平方和 (SST) :

$$SST = \sum_{j=1}^t \sum_{i=1}^{n_j} (x_{ij} - m)^2$$

推导:

$$\begin{aligned} SST &= \sum_{j=1}^t \sum_{i=1}^{n_j} (x_{ij} - m)^2 \\ &= \sum_{j=1}^t \sum_{i=1}^{n_j} [(x_{ij} - m_j) + (m_j - m)]^2 \\ &= \sum_{j=1}^t \sum_{i=1}^{n_j} [(x_{ij} - m_j)^2 + (m_j - m)^2 + 2(x_{ij} - m_j)(m_j - m)] \\ &= \sum_{j=1}^t \sum_{i=1}^{n_j} (x_{ij} - m_j)^2 + \sum_{j=1}^t \sum_{i=1}^{n_j} (m_j - m)^2 + 2 \sum_{j=1}^t \sum_{i=1}^{n_j} (x_{ij} - m_j)(m_j - m) \end{aligned}$$

分析:

- 第一项:

$$\sum_{j=1}^t \sum_{i=1}^{n_j} (x_{ij} - m_j)^2 = SSE \quad (\text{簇内平方和})$$

- 第二项:

$$\sum_{j=1}^t \sum_{i=1}^{n_j} (m_j - m)^2 = \sum_{j=1}^t n_j (m_j - m)^2 = SSB \quad (\text{簇间平方和})$$

- 第三项 (交叉项) :

$$2 \sum_{j=1}^t \sum_{i=1}^{n_j} (x_{ij} - m_j)(m_j - m) = 2 \sum_{j=1}^t (m_j - m) \sum_{i=1}^{n_j} (x_{ij} - m_j)$$

$$\sum_{i=1}^{n_j} (x_{ij} - m_j) = 0 \quad (\text{由引理: 簇内点到中心的偏差和为0})$$

$$\therefore 2 \sum_{j=1}^t (m_j - m) \sum_{i=1}^{n_j} (x_{ij} - m_j) = 0$$

结论:

$$SST = SSE + SSB + 0$$

$$SST = SSE + SSB$$

簇有效性: “Black Arts”

“The validation of clustering structures is the most difficult and frustrating part of cluster analysis. Without a strong effort in this direction, cluster analysis will remain a black art accessible only to those true believers who have experience and great courage.” 聚类结构的验证是聚类分析中最困难和最令人沮丧的部分。如果不在这方面付出巨大努力，聚类分析将仍然是一门只有那些有经验和极大勇气的真正信徒才能掌握的黑暗魔法。

Algorithms for Clustering Data, Jain and Dubes



讨论



The entire time I knew Thanos, he only ever had one goal.
To bring balance to the universe by wiping out half of all life.

Q: 灭霸一个响指就能灭掉一半生命，那怎么保证物种均衡平衡嘞~?

A: 聚类分析帮助你!

宇宙包含“九大国度”，为防止某个国度灭绝，在每个国度选取一个聚类中心，通过K-means得到聚类簇，再在每个聚类中随机选取一半生物。





讨论

Method : K-means Clustering



谢谢大家！

相关课程资源及参考文献请浏览

超算习堂: <https://easyhpc.net/course/143>